The Graph Protocol Contracts

The Graph

HALBORN

Summary

100% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ABOUT THIS REPORT

ASSESSMENT TYPE
Assurance Assessment

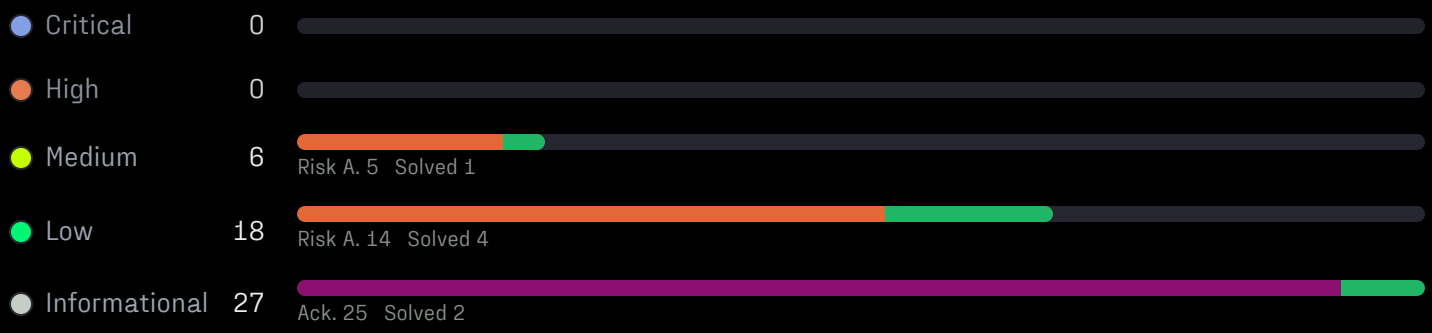
DATE OF ENGAGEMENT
Aug 4, 2026 - Aug 31, 2026

SOURCE CODE
contracts

ASSESSED COMMIT ID
78d3716

SEVERITY BREAKDOWN

51	7	19	25
All Findings	Solved	Risk Accepted	Acknowledged



Summary

`The Graph` team engaged `Halborn` to conduct a security assessment on their smart contracts beginning on August 5th 2026 and ending on August 31st, 2026. The assessment scope was limited to the smart contracts provided to `Halborn`. Commit hashes and additional details are available in the Scope section of this report.

The reviewed contracts comprise The Graph's Horizon protocol, a staking, payments and dispute layer through which indexers provision stake against data services, delegators back them, and consumers pay for indexing and query service. `HorizonStaking` custodies all staked and delegated tokens behind per verifier provisions whose unbonding deadlines follow parameters the verifier must accept and which remain exposed to verifier initiated slashing, and `RecurringCollector` meters payment against elapsed service time under agreements authorized by either a stored offer or a signature from a delegated signer, settling each collection from payer escrow; the remaining contracts handle service registration and allocation lifecycle, optimistic dispute resolution, token issuance and reward accounting, and curation signal. Authority is delegated throughout by operator and signer authorizations checked at the point of use rather than at registration, governance is split between a

self administering governor role over issuance and a single owner over the subgraph service and dispute contracts, and every component sits behind an upgradeable proxy with only protocol directory addresses and the revocation and escrow thawing bounds fixed at deployment. The acknowledged findings recorded in the repository's audits directories are not re-reported here.

ASSESSMENT SUMMARY

`Halborn` was provided with 18 days for this engagement and assigned 1 full-time security engineer to assess the security of the smart contracts in scope. The assigned engineer possesses deep expertise in blockchain and smart contract security, including hands-on experience with multiple blockchain protocols.

The objective of this assessment is to:

- Identify potential security vulnerabilities within the smart contracts.
- Verify that the smart contract functionality operates as intended.

In summary, `Halborn` identified several areas for improvement to reduce the likelihood and impact of security risks, which were addressed by `The Graph` team. The main recommendations are:

- `Bind the fisherman into the indexing and indexing fee dispute identifiers as query disputes already do, and reject an indexer disputing its own allocation.`
- `Snapshot the slashing cap at dispute creation and resolve against that stored value rather than the live parameter.`
- `Forward the indexing fee cut to an explicit destination, either forwarding it on receipt as the query fee path does or exposing an owner gated withdrawal that does not disturb that path's accounting.`

TEST APPROACH AND METHODOLOGY

`Halborn` performed a combination of manual review of the code and automated security testing to balance efficiency, timeliness, practicality, and accuracy in regard to the scope of the smart contract assessment. While manual testing is recommended to uncover flaws in logic, process, and implementation; automated testing techniques help enhance coverage of smart contracts and can quickly identify items that do not follow security best practices.

The following phases and associated tools were used throughout the term of the assessment:

- Research into the architecture and purpose of the in-scope contracts.
- Manual code review and walkthrough of the in-scope contracts.
- Manual assessment of critical `Solidity` variables and functions to identify potential vulnerability classes.
- Manual testing using custom scripts.
- Static Analysis of security for scoped contract, and imported functions (`Slither`).
- Local deployment and testing with (`Foundry`, `Remix IDE`).

Risk Methodology

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (m_e)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A)	1
	Specific (AO:S)	0.2
Attack Cost (AC)	Low (AC:L)	1
	Medium (AC:M)	0.67
	High (AC:H)	0.33
Attack Complexity (AX)	Low (AX:L)	1
	Medium (AX:M)	0.67
	High (AX:H)	0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N)	0
	Low (C:L)	0.25
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (s)	Changed (S:C)	1.25
	Unchanged (S:U)	1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score S is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

Critical	High	Medium	Low	Informational
9 - 10	7 - 8.9	4.5 - 6.9	2 - 4.4	0 - 1.9

Scope

REPOSITORY

(a) Repository: contracts

(b) Assessed Commit ID: 78d3716f0d26553e84c635b3b584945d30cc3fca

(c) Items in scope:

- packages 31 files
  - contracts/contracts 3 files
    - l2/curation 1 file
      - L2Curation.sol Solidity
    - rewards 2 files
      - RewardsManager.sol Solidity
      - RewardsManagerStorage.sol Solidity
  - horizon/contracts 9 files
    - data-service 3 files
      - extensions 1 file
        - DataServiceFees.sol Solidity
      - libraries 1 file
        - StakeClaims.sol Solidity
      - utilities 1 file
        - ProvisionManager.sol Solidity
    - payments 2 files
      - collectors 1 file
        - RecurringCollector.sol Solidity
      - PaymentsEscrow.sol Solidity
    - staking 2 files
      - HorizonStaking.sol Solidity
      - HorizonStakingBase.sol Solidity
    - utilities 2 files
      - Authorizable.sol Solidity
      - GraphDirectory.sol Solidity
  - interfaces/contracts/contracts/rewards 1 file

- RewardsCondition.sol Solidity
- issuance/contracts 8 files
  - agreement 2 files
    - RecurringAgreementHelper.sol Solidity
    - RecurringAgreementManager.sol Solidity
  - allocate 2 files
    - DirectAllocation.sol Solidity
    - IssuanceAllocator.sol Solidity
  - common 2 files
    - BaseUpgradeable.sol Solidity
    - EnumerableSetUtil.sol Solidity
  - eligibility 2 files
    - RewardsEligibilityHelper.sol Solidity
    - RewardsEligibilityOracle.sol Solidity
- subgraph-service/contracts 10 files
  - libraries 5 files
    - AllocationHandler.sol Solidity
    - IndexingAgreement.sol Solidity
    - IndexingAgreementDecoder.sol Solidity
    - IndexingAgreementDecoderRaw.sol Solidity
    - LegacyAllocation.sol Solidity
  - utilities 2 files
    - AllocationManager.sol Solidity
    - Directory.sol Solidity
  - DisputeManager.sol Solidity
  - SubgraphService.sol Solidity
  - SubgraphServiceStorage.sol Solidity

Out-of-Scope: Third party dependencies and economic attacks.

REMEDIATION COMMIT ID:

1940caf24e833227b830d975e66604d74cacd116

Out-of-Scope: New features/implementations after the remediation commit IDs.

Assessment Summary & Findings Overview

#	TITLE	SEVERITY	SCORE	STATUS
HAL-001	Indexing dispute ID does not bind the fisherman, enabling self-dispute	Medium	6.3	RISK ACCEPTED 10/09/2026

#	TITLE	SEVERITY	SCORE	STATUS
	immunization of fraudulent POIs			
HAL-002	DisputeManager maxSlashingCut Change Retroactively Affects In-Flight Disputes, Enabling Slash Cap Manipulation	Medium	5.6	RISK ACCEPTED 10/09/2026
HAL-003	Eligibility lapse permanently strands an honest indexer's earned rewards	Medium	5.4	RISK ACCEPTED 10/09/2026
HAL-004	Indexing fee data service cut is trapped in the contract	Medium	5.0	RISK ACCEPTED 10/09/2026
HAL-005	Changing the dispute period invalidates every existing provision	Medium	5.0	RISK ACCEPTED 10/09/2026
HAL-006	Stake-claim FIFO ordering and item cap can block fee collection	Medium	5.0	SOLVED 10/09/2026
HAL-007	Direct GRT transfer to curation contract permanently dilutes global reward accumulation rate for all indexers	Low	4.2	RISK ACCEPTED 10/09/2026
HAL-008	L2Curation mint() accepts zero-signal output when _signalOutMin=0, permanently donating depositor's GRT to the pool	Low	3.4	RISK ACCEPTED 10/09/2026
HAL-009	Zero-token collection advances the collection clock and permanently forfeits accrued service time	Low	3.4	RISK ACCEPTED 10/09/2026
HAL-010	Agreement update applies new pricing retroactively to the full collection window	Low	3.1	RISK ACCEPTED 10/09/2026

#	TITLE	SEVERITY	SCORE	STATUS
HAL-011	Legacy delegate() lacks slippage protection, enabling sandwich attacks via addToDelegationPool()	Low	3.1	RISK ACCEPTED 10/09/2026
HAL-012	Retroactive rate increase during pause violates the stated issuance upper bound invariant	Low	3.1	SOLVED 10/09/2026
HAL-013	Revoked collector retains just in time deposit capability and can extract the deposited funds	Low	3.1	SOLVED 10/09/2026
HAL-014	Payer-controlled eligibility callback can indefinitely withhold earned recurring fees	Low	3.1	RISK ACCEPTED 10/09/2026
HAL-015	Authorized operator can redirect an indexer's payment destination	Low	2.5	RISK ACCEPTED 10/09/2026
HAL-016	Provision parameter acceptance does not enforce the documented registration requirement	Low	2.5	SOLVED 10/09/2026
HAL-017	Agreement term validation does not bound the per entity fee rate	Low	2.5	RISK ACCEPTED 10/09/2026
HAL-018	Unbounded Stake Claims Traversal in collect() Critical Path Causes Gas DoS	Low	2.5	RISK ACCEPTED 10/09/2026
HAL-019	Permissionless Accumulator Mutators Allow Caller to Bias Reward Distribution	Low	2.5	RISK ACCEPTED 10/09/2026
HAL-020	Proof of indexing validity is never verified on chain	Low	2.5	RISK ACCEPTED 10/09/2026

#	TITLE	SEVERITY	SCORE	STATUS
HAL-021	Revoked signer addresses are permanently unusable protocol-wide	Low	2.5	SOLVED 10/09/2026
HAL-022	Self authorization override disarms the signer access modifier on three mutating functions	Low	2.5	RISK ACCEPTED 10/09/2026
HAL-023	Signer revocation thawing period accepts any value while the documented bound is enforced only on the escrow side	Low	2.5	RISK ACCEPTED 10/09/2026
HAL-024	Undelegation derives the unbonding deadline from a provision it never requires to exist	Low	2.5	RISK ACCEPTED 10/09/2026
HAL-025	Flat curve lets signal be minted and burned around a query-fee collect() to skim injected fees	Informational	1.7	ACKNOWLEDGED 10/09/2026
HAL-026	Off-by-one in RecurringAgreementManager.beforeCollection deposit condition causes JIT top-up failure when contract balance exactly equals deficit	Informational	1.7	ACKNOWLEDGED 10/09/2026
HAL-027	GraphDirectory Immutable Protocol Addresses Cannot Be Updated Post-Deployment, Creating Stale Address Risk on Protocol Contract Migration	Informational	1.7	ACKNOWLEDGED 10/09/2026
HAL-028	Integer Division Truncation in createQueryDisputeConflict Causes Permanent Token Loss of 1 Wei	Informational	1.7	ACKNOWLEDGED 10/09/2026

#	TITLE	SEVERITY	SCORE	STATUS
HAL-029	Unbounded entities Parameter Causes Overflow and DoS in IndexingAgreement._tokens ToCollect	Informational	1.7	ACKNOWLEDGED 10/09/2026
HAL-030	Issuance accrues by block.number whose semantics are chain-dependent	Informational	1.7	ACKNOWLEDGED 10/09/2026
HAL-031	EIP-712 domain separator is cached at initialization and not recomputed on chain ID change	Informational	1.7	ACKNOWLEDGED 10/09/2026
HAL-032	Missing __gap Storage Arrays in SubgraphServiceV1Storage and SubgraphServiceV2Storage Creates Storage Collision Risk on Future Upgrades	Informational	1.7	ACKNOWLEDGED 10/09/2026
HAL-033	Missing __gap Storage Array in HorizonStakingV1Storage Risks Storage Collision on Future Upgrades to the Protocol's Central Staking Contract	Informational	1.7	ACKNOWLEDGED 10/09/2026
HAL-034	A provision can be created with no unbonding period at all	Informational	1.7	ACKNOWLEDGED 10/09/2026
HAL-035	Offer cancellation is recorded under the caller while authorization is checked under the recovered signer	Informational	1.7	ACKNOWLEDGED 10/09/2026

#	TITLE	SEVERITY	SCORE	STATUS
HAL-036	DisputeManager Arbitrator Can Be Instantly Replaced With No Timelock, Enabling Single-Transaction Governance Takeover of All Dispute Resolution	Informational	1.5	ACKNOWLEDGED 10/09/2026
HAL-037	Full delegation slash strands thawing principal and blocks all delegator exits	Informational	1.4	ACKNOWLEDGED 10/09/2026
HAL-038	Expired thaw requests remain locked behind an earlier request with a later maturity	Informational	1.4	ACKNOWLEDGED 10/09/2026
HAL-039	Large eligibilityPeriod Setting Bypasses Oracle Access Control, Making All Unregistered Indexers Eligible for Rewards	Informational	1.1	ACKNOWLEDGED 10/09/2026
HAL-040	RewardsManager Subgraph Availability Oracle Can Immediately Block All Subgraph Rewards via setMinimumSubgraphSignal	Informational	1.1	ACKNOWLEDGED 10/09/2026
HAL-041	Governor role administers itself with no fallback admin and no enforced delay	Informational	1.0	ACKNOWLEDGED 10/09/2026
HAL-042	Single-Step Ownership Transfer via OwnableUpgradeable in SubgraphService and DisputeManager Enables Irreversible Governance Loss	Informational	1.0	ACKNOWLEDGED 10/09/2026
HAL-043	PAUSE_ROLE can revoke critical operational roles via emergencyRevokeRole, enabling DoS of agreement management	Informational	1.0	ACKNOWLEDGED 10/09/2026

#	TITLE	SEVERITY	SCORE	STATUS
HAL-044	Unbounded max POI staleness allows reward confiscation or freshness bypass	Informational	1.0	ACKNOWLEDGED 10/09/2026
HAL-045	Unbounded stake to fees ratio bricks fee collection	Informational	1.0	ACKNOWLEDGED 10/09/2026
HAL-046	NatSpec Unit Mismatch in Allocation.isStale() Documents staleThreshold as Blocks but Compares Against Seconds, Risking Governance Misconfiguration	Informational	0.8	SOLVED 10/09/2026
HAL-047	Missing maxInitialTokens Overflow Validation in RecurringCollector Allows Permanent DoS on First Collection	Informational	0.5	ACKNOWLEDGED 10/09/2026
HAL-048	Eligibility parameter setters lack upper bounds causing potential revert in isEligible	Informational	0.5	ACKNOWLEDGED 10/09/2026
HAL-049	Governor role is self-administering with no enforced timelock or separation	Informational	0.5	ACKNOWLEDGED 10/09/2026
HAL-050	Payer cancellation of an accepted agreement is a request to the data service rather than a state change	Informational	0.5	SOLVED 10/09/2026
HAL-051	Thawed token view and the executable fulfillment path order maturity and nonce checks differently	Informational	0.5	ACKNOWLEDGED 10/09/2026

Findings & Tech Details

HAL-001 RISK ACCEPTED

Indexing dispute ID does not bind the fisherman, enabling self-dispute immunization of fraudulent POIs

Medium · 6.3 AO:A/AC:M/AX:L/R:N/S:U/C:N/A:H/I:H/D:N/Y:N

DESCRIPTION

Query dispute identifiers in `DisputeManager._createQueryDisputeWithAttestation()` include the fisherman address in the hash preimage, so two fishermen disputing the same attestation produce distinct identifiers and can coexist. `DisputeManager._createIndexingDisputeWithAllocation()` and `DisputeManager._createIndexingFeeDisputeV1()` omit it, deriving the identifier from the allocation, POI and block number in the first case and from a literal, the agreement id, POI, entity count and block number in the second.

SOL 1-9

```
bytes32 disputeId = keccak256(
    abi.encodePacked(
        _attestation.requestCID,
        _attestation.responseCID,
        _attestation.subgraphDeploymentId,
        indexer,
        _fisherman          // address bounded
    )
);
```

SOL 1-7

```
// _createIndexingDisputeWithAllocation()
bytes32 disputeId = keccak256(abi.encodePacked(_allocationId, _poi, _blockNumber));

// _createIndexingFeeDisputeV1()
bytes32 disputeId = keccak256(
    abi.encodePacked("IndexingFeeDisputeWithAgreement", _agreementId, _poi, _entities,
_blockNumber)
);
```

Both preimages are public, so any observer of a pending dispute transaction can reconstruct the identifier and submit an identical dispute ahead of the discovering fisherman. `DisputeManager.isDisputeCreated()` then rejects the original as already created. The observer inherits the fisherman position, and with it the deposit refund and the reward paid in

`DisputeManager._acceptDispute()`. The indexer's exposure is unchanged, but the incentive to detect fraud is transferred away from whoever did the work, which is the only reason the fisherman role exists.

A second consequence is that the slot for a given tuple can be occupied deliberately. Terminal resolutions in `DisputeManager._acceptDispute()`, `_rejectDispute()`, `_drawDispute()` and `_cancelDispute()` set a status but never clear the entry, and `_createIndexingDisputeWithAllocation()` neither checks that the POI was actually presented nor prevents the allocation's own indexer from being the fisherman. An indexer can therefore file against a tuple it intends to use, wait out the dispute period with nothing for the arbitrator to evaluate, cancel through `DisputeManager.cancelDispute()` for a full refund, and only then present the POI.

This raises the cost of disputing that exact tuple but does not eliminate it. The block number argument is unconstrained by any on chain check, so an honest fisherman can file the substantively identical dispute under a neighbouring block number and the arbitrator evaluates the same evidence. A squatter would have to occupy every candidate value, paying a deposit and a full dispute period for each. Treat this as friction added to the dispute path rather than immunity.

That friction matters more here than it would in a system with on chain verification. `AllocationHandler.presentPOI()` accepts any value as a proof of indexing and uses it only to separate the zero POI reclaim path from the reward path. It checks staleness, maturity and subgraph denial, but never verifies that the submitted proof corresponds to correct indexing work, and when the conditions pass `RewardsManager.takeRewards()` mints based on allocation size and duration alone. Correctness rests entirely on the optimistic dispute game, so anything that raises the cost or lowers the payoff of disputing weakens the only check on fraudulent reward claims.

RECOMMENDATION

Include the caller in the identifier derivation for both `DisputeManager._createIndexingDisputeWithAllocation()` and `DisputeManager._createIndexingFeeDisputeV1()`, matching the pattern already used for query disputes. This removes the reward capture vector and lets independent fishermen file without colliding. Separately, add a creation time check that the fisherman is not the disputed allocation's own indexer, and consider clearing the mapping entry in `_cancelDispute()` and `_drawDispute()` so a tuple becomes disputable again after a resolution that reached no conclusion.

SOL

```
bytes32 disputeId = keccak256(abi.encodePacked(_allocationId, _poi, _blockNumber, _fisherman));
```

REMEDIATION COMMENT

RISK ACCEPTED: The fisherman-independent indexing dispute identifiers were retained by the `The Graph` team. Duplicate identifiers are still rejected in `DisputeManager.sol:441-444` and `DisputeManager.sol:516-521`, while the stored fisherman receives the reward in `DisputeManager.sol:560-565`. The reported identifier collision remains. The following statement was provided by the `The Graph` team:

> Indexing dispute IDs intentionally exclude the fisherman to enforce on-chain that an indexer can be disputed at most once per indexing fault (double jeopardy protection, see OZ 2025 Horizon audit L-09). Including the fisherman would allow unlimited disputes for the same fault.
>
> Front-running is impractical on Arbitrum's private FCFS sequencer and would only change who receives the fisherman reward, not whether the indexer is slashed.
>
> The squatting scenario is not effective. A dispute cannot be cancelled before the dispute period ends, and until then the arbitrator can resolve it. Self-disputes, including ones filed through an address linked to the indexer, are handled by the arbitrator under the Arbitration Charter. `blockNumber` is also unconstrained, so an honest fisherman can always file an equivalent dispute. For the same reason, an on-chain `fisherman != indexer` check would add no protection: the indexer could use any other address.

HAL-002 RISK ACCEPTED

DisputeManager maxSlashingCut Change Retroactively Affects In-Flight Disputes, Enabling Slash Cap Manipulation

Medium · 5.6 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:M/I:L/D:N/Y:N

DESCRIPTION

The `DisputeManger._slashIndexer()` function reads the current `maxSlashingCut` at dispute resolution time rather than the value that was in effect when the dispute was created. The `stakeSnapshot` is captured at creation time and stored in the `Dispute` struct, but `maxTokensSlash = _tokensStakeSnapshot.mulPPM(maxSlashingCut)` uses the live `maxSlashingCut` state variable. This means the owner can change `maxSlashingCut` between dispute creation and resolution, retroactively altering the maximum slashable amount for all pending disputes. Increasing `maxSlashingCut` after dispute creation allows the arbitrator to slash indexers by a larger percentage than they could have anticipated at dispute filing time, violating the protocol's implicit guarantee that slashing limits are fixed at dispute creation. Decreasing `maxSlashingCut` causes the arbitrator's pre-computed `tokensSlash` (which was valid at creation) to fail with `DisputeManagerInvalidTokensSlash`, blocking dispute resolution entirely.

SOL 612-636

```
612 function _slashIndexer(
613     address _indexer,
614     uint256 _tokensSlash,
615     uint256 _tokensStakeSnapshot
616 ) private returns (uint256) {
617     ISubgraphService subgraphService_ = _getSubgraphService();
618     // Get slashable amount for indexer
619     IHorizonStaking.Provision memory provision = _graphStaking().getProvision(_indexer,
620 address(subgraphService_));
    // Ensure slash amount is within the cap
621     uint256 maxTokensSlash = _tokensStakeSnapshot.mulPPM(maxSlashingCut);
622     require(
623         _tokensSlash != 0 && _tokensSlash <= maxTokensSlash,
        DisputeManagerInvalidTokensSlash(_tokensSlash, maxTokensSlash)
624     );
625
626     //...
    uint256 maxRewardableTokens = Math.min(_tokensSlash, provision.tokens);
627     uint256 effectiveCut = Math.min(provision.maxVerifierCut, fishermanRewardCut);
628     uint256 tokensRewards = effectiveCut.mulPPM(maxRewardableTokens);
629
630     subgraphService_.slash(_indexer, abi.encode(_tokensSlash, tokensRewards));
    return tokensRewards;
631 }
```

RECOMMENDATION

Snapshot `maxSlashingCut` at dispute creation: add `maxSlashingCut` to the `Dispute` struct and record it when the dispute is created. Use this snapshot value in `_slashIndexer()`.

REMEDIATION COMMENT

RISK ACCEPTED: The live slashing cap was retained by the `The Graph` team. A cap change is applied through `DisputeManager.sol:302-303`, and the current cap is used at resolution in `DisputeManager.sol:623-627`. An in-flight dispute has no cap snapshot. The following statement was provided by the `The Graph` team:

> ̀maxSlashingCut ̀is a governance-controlled upper bound, not a per-dispute commitment. The slash amount is chosen by the arbitrator at resolution time and passed as a parameter to àcceptDispute;̀ nothing is pre-computed, so lowering the cap never blocks resolution — the arbitrator simply provides an amount within the current bound. Raising the cap requires an owner action, and the owner is already trusted with broader powers (contract upgrades, arbitrator replacement). The stake snapshot exists to prevent the disputed indexer from escaping slashing by reducing stake, which is a different threat model. Changes to ̀maxSlashingCut ̀are expected to be infrequent governance actions coordinated with the arbitration process.

HAL-003 RISK ACCEPTED

Eligibility lapse permanently strands an honest indexer's earned rewards

● Medium · 5.4 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:L/I:N/D:N/Y:H

DESCRIPTION

When an eligibility oracle is configured and `revertOnIneligible` is left at its default of false in `RewardsManager`, indexing rewards accrued over a valid allocation are lost to the indexer if the indexer is ineligible at the moment of POI presentation.

`AllocationHandler.presentPOI()` classifies a presentation into one of four conditions. Two of them, allocation too young and subgraph denied, return early and preserve the accrual for a later collection. The other two, stale POI and zero POI, route to `RewardsManager.reclaimRewards()`, which redirects the accrual rather than preserving it. Indexer ineligibility is not one of the four. It is evaluated one layer down, inside `RewardsManager.takeRewards()` by way of `RewardsManager._deniedRewards()`.

So when the indexer is ineligible but the POI is valid, the subgraph is not denied and the allocation is mature, the condition resolves to none and execution falls through to `RewardsManager.takeRewards()`. With `revertOnIneligible` false, `_deniedRewards()` passes its require and calls `_reclaimRewards()` with the indexer ineligible reason, which mints to the configured reclaim address if one exists and otherwise returns zero without minting anything. Either way `takeRewards()` returns zero to the caller. Execution then continues to `AllocationHandler.snapshotRewards()` and `AllocationHandler.clearPendingRewards()`, which advance the accumulator and clear the pending accrual for the elapsed period. The indexer's earnings for that period are gone, with no path to restore them once eligibility returns.

SOL 780–805

```
780 function _deniedRewards(
781     uint256 rewards,
782     address indexer,
783     address allocationID,
784     bytes32 subgraphDeploymentID
785 ) private returns (bool denied) {
786     bool isDeniedSubgraph = isDenied(subgraphDeploymentID);
787     bool isIneligible = address(rewardsEligibilityOracle) != address(0) &&
        !rewardsEligibilityOracle.isEligible(indexer);
788
789     // When configured to revert, block collection so rewards remain claimable if
790     // the indexer becomes eligible and collects before the allocation goes stale.
        require(!isIneligible || !revertOnIneligible, "Indexer not eligible for rewards");
791
        if (!isDeniedSubgraph && !isIneligible) return false;
792
```

```
793     if (isDeniedSubgraph) emit RewardsDenied(indexer, allocationID);
794     if (isIneligible) emit RewardsDeniedDueToEligibility(indexer, allocationID, rewards);
795
796     bytes32 reason = isDeniedSubgraph ? RewardsCondition.SUBGRAPH_DENIED :
    RewardsCondition.NONE;
797     if (isIneligible && (!isDeniedSubgraph || reclaimAddresses[reason] == address(0)))
            reason = RewardsCondition.INDEXER_INELIGIBLE;

798     _reclaimRewards(reason, rewards, indexer, allocationID, subgraphDeploymentID);
799     return true;
    }
800
```

Eligibility can flip to false on a liveness lapse rather than misbehaviour: an oracle that misses a renewal inside the eligibility window because of an outage, congestion or a transient scoring error. The magnitude is unbounded per indexer, since a large allocation forfeits a proportionally large accrual.

The safe handling exists and is deliberate. The require in `_deniedRewards()` carries a comment stating that reverting is meant to block collection so the rewards remain claimable if the indexer becomes eligible and collects before the allocation goes stale. That is the correct behaviour, and it is available only when an operator opts in by setting `revertOnIneligible` to true. The default configuration silently strands honest earnings.

RECOMMENDATION

Make the safe behaviour the default rather than an opt in, either by defaulting `revertOnIneligible` to true or by routing indexer ineligibility through the same early return that `AllocationHandler.presentPOI()` already uses for the subgraph denied condition, so a presentation blocked for ineligibility preserves the accrual instead of snapshotting and clearing it. The accrual for the affected period must remain recoverable once eligibility is restored.

REMEDIATION COMMENT

RISK ACCEPTED: The existing eligibility modes were retained by `The Graph` team. In `RewardsManager.sol:790-804`, an ineligible collection is reverted only when `revertOnIneligible` is true; otherwise the rewards are reclaimed or dropped. After a successful presentation, the allocation is snapshotted and cleared in `AllocationHandler.sol:362-372`. The stated live setting was not independently read. The following statement was provided by `The Graph` team:

> The reclaim-on-ineligible behavior (`revertOnIneligible = false`) is an intentional, documented mode for denying rewards to ineligible indexers. The protocol is deployed with `revertOnIneligible = true`: it is the default in the deployment configuration, it is applied in the GIP-0088 governance batch, and it is verified on-chain by the deployment end checks. In this mode `takeRewards` reverts for ineligible indexers and accrued rewards are preserved until eligibility is restored (subject to POI staleness). Oracle liveness failures are also mitigated by the REO fail-open: with the default parameters

> (òracleUpdateTimeout`of 7 days, shorter than the 14-day èligibilityPeriod), an oracle outage makes all indexers eligible before any renewed indexer's window expires.

HAL-004 RISK ACCEPTED

Indexing fee data service cut is trapped in the contract

Medium · 5.0 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:M/Y:N

DESCRIPTION

`SubgraphService.collect()` passes the configured cut through to the recurring collector when collecting an agreement in `IndexingAgreement.collect()`. `GraphPayments.collect()` then delivers that portion to the data service address, which is `SubgraphService` itself, as a plain transfer. Every other portion of the same payment is handled: the protocol cut is burned, the delegation cut goes through `addToDelegationPool()`, and the receiver's remainder is staked through `stakeTo()` or pushed to the receiver destination. Only the data service cut arrives with nothing to receive it. The query fee path handles its equivalent cut correctly, measuring the balance received for curation and forwarding it to the curation contract. The indexing fee path performs no measurement and no forwarding, so the cut accumulates.

SOL 773-803

```
773 //SubgraphSevice._collectIndexingFees()
774 function _collectIndexingFees(
775     address _indexer,
776     bytes16 _agreementId,
777     address _paymentsDestination,
778     bytes memory _data
779 ) private returns (uint256) {
780     (address indexer, uint256 tokensCollected) =
    IndexingAgreement._getStorageManager().collect(
781         _allocations,
782         IndexingAgreement.CollectParams({
783             indexer: _indexer,
784             agreementId: _agreementId,
785             currentEpoch: _graphEpochManager().currentEpoch(),
                receiverDestination: _paymentsDestination,
786             data: _data,
787             indexingFeesCut: indexingFeesCut
788         })
789     );
790
791     _releaseStake(indexer, 0);
792     if (tokensCollected > 0) {
793         // lock stake as economic security for fees
794         _lockStake(
795             indexer,
796             tokensCollected * stakeToFeesRatio,
797             block.timestamp + _disputeManager().getDisputePeriod()
798         );
    }
799
800     return tokensCollected;
801 }
802
```

Nothing in scope can move it. The only outflows from `SubgraphService` are bounded to amounts produced in the same call, the measured curation forwarding and the freshly minted indexing rewards, and no rescue or withdrawal function exists in the contract or its bases. Two conditions bound this. The cut is not set during initialization and defaults to zero, so nothing accrues until the owner enables it.

RECOMMENDATION

Either forward the received balance to its intended recipient inside `SubgraphService.collect()`, mirroring the query fee path, or add an owner gated withdrawal. If a withdrawal is added, keep it independent of the balance measurement the query fee path relies on.

REMEDIATION COMMENT

RISK ACCEPTED: The configurable indexing fee cut was retained by the `The Graph` team. The cut is forwarded to collection in `SubgraphService.sol:780-789` and `IndexingAgreement.sol:563-577`, but no recipient or withdrawal path is present in `SubgraphService.sol:792-803`. A nonzero cut can still be accrued in the service. The following statement was provided by the `The Graph` team:

> Confirmed. The configurable îndexingFeesCut`was introduced to allow a future non-zero data service cut without redeploying (TRST-R-6), but its destination has not been defined. The cut is currently zero and there is no plan to enable it. Governance will not set a non-zero value until an upgrade defines where the data service's share is forwarded. Any tokens accidentally accrued would be recoverable through a contract upgrade.

HAL-005 RISK ACCEPTED

Changing the dispute period invalidates every existing provision

Medium · 5.0 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:M/I:N/D:N/Y:N

DESCRIPTION

The accepted thawing period range for provisions is pinned to a single value. `SubgraphService` returns the current dispute period as both the minimum and the maximum of the range, so a provision is valid only when its thawing period is exactly equal to the current dispute period.

The instant governance changes the dispute period, every existing provision falls out of range. Every function that requires a valid provision then reverts, including `SubgraphService.collect()`, so indexers cannot withdraw already earned query fees or indexing fees until each of them individually re stages provision parameters through the staking contract and has them accepted.

The same class of problem applies to the verifier cut range, whose lower bound is pinned to the fisherman reward cut. Raising the fisherman reward cut invalidates provisions whose maximum verifier cut is below the new value.

SOL 274–299

```
274 function collect(
275     address indexer,
276     IGraphPayments.PaymentTypes paymentType,
277     bytes calldata data
278 ) external override enforceService(indexer, VALID_PROVISION | REGISTERED) returns (uint256) {
    uint256 paymentCollected = 0;
279
280     if (paymentType == IGraphPayments.PaymentTypes.QueryFee) {
281         paymentCollected = _collectQueryFees(indexer, data);
    } else if (paymentType == IGraphPayments.PaymentTypes.IndexingRewards) {
282         paymentCollected = _collectIndexingRewards(indexer, data);
283     } else if (paymentType == IGraphPayments.PaymentTypes.IndexingFee) {
        (bytes16 agreementId, bytes memory iaCollectionData) =
284 IndexingAgreementDecoder.decodeCollectData(data);
        paymentCollected = _collectIndexingFees(
285             indexer,
            agreementId,
286             paymentsDestination[indexer],
            iaCollectionData
287         );
288     } else {
289         revert SubgraphServiceInvalidPaymentType(paymentType);
290     }
291
292     emit ServicePaymentCollected(indexer, paymentType, paymentCollected);
293     return paymentCollected;
294 }
```

RECOMMENDATION

Widen the accepted thawing period range so a change to the dispute period does not immediately invalidate live provisions. A grace window that accepts both the previous and the current dispute period during a transition would avoid the stall. Apply the same treatment to the verifier cut range.

REMEDIATION COMMENT

RISK ACCEPTED: The exact provision thawing period requirement was retained by the `The Graph` team. The allowed range remains equal to the current dispute period in `SubgraphService.sol:574-576`, and old parameters fail the range check in `ProvisionManager.sol:252-262` after a change. Validity can be restored through staging and acceptance in `HorizonStaking.sol:181-230`. The following statement was provided by the `The Graph` team:

> The exact thawing period range (`min = max = disputePeriod`) was introduced deliberately to resolve OZ 2025 M-01. A thawing period shorter than the dispute period lets an indexer exit before a dispute is resolved. A longer one lets an indexer lock delegators' thaw requests. A grace window that accepts the previous dispute period would reintroduce the first issue whenever the dispute period is increased.
>
> The effect of a parameter change is temporary and does not cause loss of funds. Indexers or their operators restore validity in two transactions (`setProvisionParameters` followed by `acceptProvisionPendingParameters`). This path does not require a valid provision and has no waiting period. Outstanding fees remain collectable afterwards.
>
> Changes to `disputePeriod` or `fishermanRewardCut` are infrequent governance actions and will be announced and coordinated with indexers ahead of time.

HAL-006 SOLVED

Stake-claim FIFO ordering and item cap can block fee collection

Medium · 5.0 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:M/I:N/D:N/Y:N

DESCRIPTION

Stake claim release in `StakeClaims` is ordered by creation rather than by expiry, and the underlying list is capped at `MAX_ITEMS`, which `LinkedList` sets to 10000. Two conditions follow. First, `DataServiceFees._releaseStake()` traverses from the head and `StakeClaims.processStakeClaim()` returns a stop signal on the first claim whose release time has not passed. A head claim with a longer remaining lock therefore holds back later claims that have already expired, and pagination does not help because traversal always starts at the head. The inversion arises when the owner lowers the dispute period between claim creations, since the release time is computed as the current timestamp plus the dispute period at lock time, making a later claim releasable before an earlier one.

SOL 142–166

```
142 function processStakeClaim(
143     mapping(address serviceProvider => uint256 tokens) storage feesProvisionTracker,
    mapping(bytes32 claimId => StakeClaim claim) storage claims,
144     bytes32 _claimId,
    bytes memory _acc
145 ) external returns (bool, bytes memory) {
146     StakeClaim memory claim = claims[_claimId];
147     require(claim.createdAt != 0, StakeClaimsClaimNotFound(_claimId));
148
149     // early exit
    if (claim.releasableAt > block.timestamp) {
150         return (true, LinkedList.NULL_BYTES);
151     }
152
153     // decode
154     (uint256 tokensClaimed, address serviceProvider) = abi.decode(_acc, (uint256, address));
155
156     // process
157     feesProvisionTracker.release(serviceProvider, claim.tokens);
    emit StakeClaimReleased(serviceProvider, _claimId, claim.tokens, claim.releasableAt);
158
159     // encode
160     _acc = abi.encode(tokensClaimed + claim.tokens, serviceProvider);
    return (false, _acc);
161 }
```

Second, every `SubgraphService.collect()` locks a new claim, so sustained collection at high frequency can fill the list. Once full, `LinkedList.addTail()` reverts, and the collection reverts with it.

Both `_collectQueryFees()` and `_collectIndexingFees()` call `_releaseStake(_indexer, 0)` before collecting, and passing zero makes the traversal run to the end of the list rather than a fixed batch. This is a gas spike rather than a lockout: traversal still stops at the first unexpired claim, so it only ever walks the expired prefix, and `DataServiceFees.releaseStake(uint256)` is externally callable and paginated, so an indexer facing a large expired batch can clear it in chunks and then collect. The NatSpec on both collection paths states this workaround. It remains an unbounded loop on the critical path, and the recovery depends on the indexer knowing to perform a manual step.

Both conditions are largely self inflicted, since the indexer's own collection cadence creates the claims, and both are temporary, since collection resumes once the blocking claims expire and are released. The head blocking inversion specifically requires a dispute period reduction; absent such a change, claims expire in the order they were created and the head is always the earliest to release.

RECOMMENDATION

Allow expired claims to be released independently of a not-yet-expired head; for example, by releasing per claim or ordering release by expiry time rather than creation order. Additionally, prevent a full list from permanently blocking `SubgraphService.collect()`, either by auto-releasing expired claims before locking a new one or by bounding claim growth so the cap cannot be reached during normal collection.

REMEDIATION COMMENT

Addressed in: 1940caf24e833227b830d975e66604d74cacd116

SOLVED: The stake-claim release behavior was documented by the `The Graph` team in `IDataServiceFees.sol:28-40`. Limitation: The runtime still stops at an unexpired head in `StakeClaims.sol:142-165`, and the 10,000-item cap remains in `LinkedList.sol:34-53`. Fee collection can still be delayed until claims can be released. The following statement was provided by the `The Graph` team:

> The head-of-line blocking after a dispute period reduction was previously reported as OZ 2025 L-02, which covered both thaw requests and stake claims. The team decided to document the behavior instead of reworking the ordering, which would add significant complexity. The blockage is temporary, only occurs after a governance reduction of the dispute period, and does not cause loss of funds. We will extend the documentation to `DataServiceFees.releaseStake`.
>
> Reaching `MAX_ITEMS` requires a single indexer to perform more than 10,000 fee collections within one dispute period, because expired claims are released before each new claim is locked. Collection frequency is fully under the indexer's control: RAVs aggregate receipts, and partial collection is supported. Collection resumes as claims expire. Large expired batches can be released in chunks through the paginated `releaseStake(n)`, as documented on both collection paths.

HAL-007 RISK ACCEPTED

Direct GRT transfer to curation contract permanently dilutes global reward accumulation rate for all indexers

Low · 4.2 AO:A/AC:M/AX:L/R:N/S:U/C:N/A:N/I:M/D:N/Y:M

DESCRIPTION

The `RewardsManager._getNewRewardsPerSignal()` function calculates the global GRT reward accumulation rate by dividing total issuance per block by `graphToken().balanceOf(address(curation()))`, which is the total GRT token balance held by the curation contract. However, the curation contract's signal via `getCurationPoolTokens(id)` tracks per-subgraph `pools[id].tokens`, and these pool-level counters are only updated through the official curation operations mint , burn , collect). Any GRT tokens directly transferred to the curation contract (without calling an official curation function) inflate any `balanceOf(curation)` without updating `pools[id].tokens`.

This creates a griefing attack vector: an attacker can directly transfer any amount of GRT to the curation contract address to permanently inflate the denominator in `_getNewRewardsPerSignal()`, reducing `accRewardsPerSignal` accumulation for all future blocks. This reduces every indexer's reward share proportionally, and since accRewardsPerSignal never decreases, historical accumulations are unaffected but future accumulations are diluted indefinitely. The phantom signal tokens cannot be claimed by any indexer as they are not allocated to any subgraph pool

RECOMMENDATION

Derive the reward per signal denominator from an internally accounted total of curated pool tokens rather than from `graphToken().balanceOf(address(curation()))`. The cleanest form is a total signalled tokens accessor on the curation contract, maintained alongside `pools[id].tokens`; an alternative is a counter kept in `RewardsManager` and updated through the existing `onSubgraphSignalUpdate()` and `onSubgraphAllocationUpdate()` hooks. Once the denominator cannot be moved by an unsolicited transfer, the instant of a segment close becomes value neutral and the ungated mutators need no further treatment. If the denominator is left as a balance, make the per signal increment time weighted so the cumulative integral does not depend on when segments were closed.

REMEDIATION COMMENT

RISK ACCEPTED: The balance-based reward denominator was retained by `The Graph` team. Accrued issuance is still divided by the curation contract's token balance in `RewardsManager.sol:372-390`. A

direct token transfer is therefore included in the denominator without an increase in accounted pool signal. The following statement was provided by `The Graph` team:

> Using the curation contract's token balance as the rewards-per-signal denominator is a long-standing, known design choice of the protocol. Exploiting it requires an attacker to permanently lock GRT in the curation contract with no gain: the diluted share of issuance is simply not minted rather than redirected. This makes it an economic griefing vector, which is out of scope per the engagement terms.

HAL-008 RISK ACCEPTED

L2Curation mint() accepts zero-signal output when _signalOutMin=0, permanently donating depositor's GRT to the pool

Low · 3.4 AO:A/AC:M/AX:L/R:N/S:U/C:N/A:N/I:N/D:M/Y:N

DESCRIPTION

The `L2Curation.mint()` function does not explicitly reject a computed `signalOut` of zero. The only guard is `require(signalOut >= _signalOutMin)`, which passes trivially when `_signalOutMin = 0` (the default minimum). After pool initialization by a first depositor who deposits exactly `minimumCurationDeposit` tokens, the pool holds approximately `minimumCurationDeposit` tokens but only 1 wei `SIGNAL_PER_MINIMUM_DEPOSIT = 1`) of GCS signal. Any subsequent depositor whose `_tokensIn` is less than `curationPool.tokens` (≈ `minimumCurationDeposit`) will compute `signalOut = 1 * _tokensIn / curationPool.tokens = 0` (integer division rounds down). With `_signalOutMin = 0`, the function proceeds to pull GRT from the depositor, burn the curation tax, add the net tokens to the pool, and call `gcs.mint(curator, 0)`, permanently donating the depositor's GRT to the pool without issuing any signal token in return.

RECOMMENDATION

Add an explicit non-zero signal check in `mint()` and `mintTaxFree()` which has the same vulnerability. For the first depositor: Instead of `SIGNAL_PER_MINIMUM_DEPOSIT = 1`, use a larger value (e.g., `1e18`) to reduce the ratio sensitivity and make zero-signal conditions practically impossible for reasonable deposit amounts.

REMEDIATION COMMENT

RISK ACCEPTED: The zero-signal mint behavior was retained by `The Graph` team. In `L2Curation.sol:203-239`, zero signal is still accepted when a zero minimum is supplied, and the input tokens are then pulled. No nonzero guard was added to tax-free minting in `L2Curation.sol:249-287`. The configured 1-wei minimum applies to an empty pool in `L2Curation.sol:492-505`; zero signal can still be produced by rounding in a funded pool. The following statement was provided by `The Graph` team:

> On Arbitrum One `minimumCurationDeposit` is 1 wei, so the first deposit mints signal close to 1:1 with tokens and the described rounding scenario does not arise under normal operation. Curators minting directly are protected by `_signalOutMin,` and GNS minting is protected by `nSignalOutMin.` We acknowledge that an explicit zero-signal check would add defense in depth.

HAL-009 RISK ACCEPTED

Zero-token collection advances the collection clock and permanently forfeits accrued service time

Low · 3.4 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:N/D:M/Y:N

DESCRIPTION

`RecurringCollector._collect()` updates `lastCollectionAt` to the current block timestamp unconditionally, before any check on whether tokens actually moved. When the requested amount resolves to zero, no escrow transfer occurs and neither `_preCollectCallbacks()` nor `_postCollectCallback()` runs, since both are gated on a nonzero collection amount. The clock still advances.

SOL 1-86

```
function _collect(
  IGraphPayments.PaymentTypes _paymentType,
  CollectParams memory _params
) private returns (uint256) {
  AgreementData storage agreement = _getAgreementStorage(_params.agreementId);

  // Check if agreement is collectable first
  (bool isCollectable, uint256 collectionSeconds, AgreementNotCollectableReason reason) =
_getCollectionInfo(
      agreement
  );
  require(isCollectable, RecurringCollectorAgreementNotCollectable(_params.agreementId,
reason));

  require(
      msg.sender == agreement.dataService,
      RecurringCollectorDataServiceNotAuthorized(_params.agreementId, msg.sender)
  );

  // Check the service provider has an active provision with the data service
  // This prevents an attack where the payer can deny the service provider from collecting
payments
  // by using a signer as data service to syphon off the tokens in the escrow to an account
they control
  {
      uint256 tokensAvailable = _graphStaking().getProviderTokensAvailable(
          agreement.serviceProvider,
          agreement.dataService
      );
      require(tokensAvailable > 0,
RecurringCollectorUnauthorizedDataService(agreement.dataService));
  }

  // Always validate temporal constraints (min/maxSecondsPerCollection) even for
  // zero-token collections, to prevent bypassing temporal windows while updating
  // lastCollectionAt.
  uint256 tokensToCollect = _requireValidCollect(
      agreement,
      _params.agreementId,
      _params.tokens,

```

```
        collectionSeconds
    );

    if (_params.tokens != 0) {
        uint256 slippage = _params.tokens - tokensToCollect;
        require(
            slippage <= _params.maxSlippage,
            RecurringCollectorExcessiveSlippage(_params.tokens, tokensToCollect,
_params.maxSlippage)
        );
    }
    agreement.lastCollectionAt = uint64(block.timestamp);

    address payer = agreement.payer;

    if (0 < tokensToCollect) {
        _preCollectCallbacks(agreement, _params.agreementId, tokensToCollect);

        _graphPaymentsEscrow().collect(
            _paymentType,
            payer,
            agreement.serviceProvider,
            tokensToCollect,
            agreement.dataService,
            _params.dataServiceCut,
            _params.receiverDestination
        );
    }

    emit PaymentCollected(
        _paymentType,
        _params.collectionId,
        payer,
        agreement.serviceProvider,
        agreement.dataService,
        tokensToCollect
    );

    emit RCACollected(
        agreement.dataService,
        payer,
        agreement.serviceProvider,
        _params.agreementId,
        _params.collectionId,
        tokensToCollect,
        _params.dataServiceCut
    );

    if (0 < tokensToCollect)
        _postCollectCallback(payer, agreement.conditions, _params.agreementId,
tokensToCollect);
    return tokensToCollect;
}

```

Because the collection window is derived from `lastCollectionAt`, every service second accrued since the previous collection is discarded. The next collection can only claim time measured from the zero token call forward, and the forfeited period is unrecoverable.

The reachable actor is narrower than the entry point suggests and worth naming precisely. Only the agreement's data service may call `RecurringCollector.collect()`, and for the deployed data

service that call arrives through `SubgraphService.collect()`, which is gated on the indexer or an operator the indexer has authorized on this verifier. The requested amount is then computed by `IndexingAgreement._tokensToCollect()` from the agreement terms and the entity count the caller supplies, so a zero request means the caller supplied terms and an entity count that produce zero. In practice this is either the indexer harming itself or an authorized operator griefing the indexer, which places it alongside the payment destination finding as an abuse of the operator role rather than an external attack.

Temporal validation still applies, so the call is bounded to once per minimum collection interval. That bound is enough to sustain the effect indefinitely: an operator repeating zero token collections at the minimum interval holds the provider's claimable window at the minimum on every subsequent collection, capping earnings well below the agreed terms, without transferring any tokens and without notifying the payer through any callback.

The interaction was partly considered. The contract documents that temporal validation runs even for zero token collections, to prevent bypassing collection windows while updating `lastCollectionAt`. The timing half of that interaction is guarded; the forfeiture of the accrued window is not.

RECOMMENDATION

The collection clock should advance only when a collection actually settles value, or a zero-token collection should preserve the accrued window rather than resetting the reference point. If advancing the clock on a zero-token call is required to keep temporal windows from being bypassed, the accrued period should be carried forward so it remains claimable on the next collection instead of being discarded.

REMEDIATION COMMENT

RISK ACCEPTED: Zero-token clock advancement was retained by the `The Graph` team. The collection clock was verified to advance before the positive-token transfer guard in `RecurringCollector.sol:742-765`; accrued service time can still be forfeited. The following statement was provided by the `The Graph` team:

> Only the data service can collect, and in SubgraphService only the indexer or an operator it has authorized can trigger a collection. The scenario therefore requires a malicious authorized operator. Operators are trusted by the indexer to manage allocations and collections on its behalf. Such an operator could equally forfeit most of the window with a small non-zero collection, for example by under-reporting èntities,` so gating the clock update on a non-zero amount would not remove the underlying trust assumption. Advancing `lastCollectionAt` on zero-token collections is intentional, and temporal constraints are enforced for these calls. Indexers should only authorize trusted operators and can revoke them at any time.

HAL-010 RISK ACCEPTED

Agreement update applies new pricing retroactively to the full collection window

Low · 3.1 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:L/Y:N

DESCRIPTION

When `RecurringCollector.update()` is called, `_validateAndStoreUpdate()` overwrites the maximum ongoing rate and the other terms but leaves `lastCollectionAt` untouched. On the next call to `RecurringCollector.collect()`, `_requireValidCollect()` computes the ceiling as the updated rate multiplied by the collection seconds, where the collection seconds reach back to `lastCollectionAt` and therefore span time that elapsed before the update was applied. The requested amount on the data service side behaves the same way, since `IndexingAgreement._tokensToCollect()` also multiplies the updated per second terms by the full window. A rate increase applied mid window bills the entire window at the new rate.

Two things bound this. The behaviour is documented at the entry point: the NatSpec on `RecurringCollector.update()` states that updated pricing terms apply immediately and affect the next collection for the entire period since `lastCollectionAt`. And the update is authorized by the payer, either through a signature over the update or through a stored offer the payer registered, so the party that bears the cost is the party that approves the change.

What remains is that the semantics are not visible in the signed artefact. An update carries an end time, per second rates and a nonce, but nothing that expresses an effective time, so a payer reasoning about the message rather than about the collector's implementation would reasonably expect the new rate to apply from the update forward.

RECOMMENDATION

Record an effective timestamp with the rate update and apply the new rate in `RecurringCollector._requireValidCollect()` only to seconds elapsed after that timestamp, charging the preceding portion of the window at the previous rate. If the current behaviour is intended, surface it in the signed artefact rather than only in the contract documentation, so a payer approving an update can see the window it applies to.

REMEDIATION COMMENT

RISK ACCEPTED: Retroactive application of updated pricing was retained by the `The Graph` team. The updated rate was verified to replace the prior terms without a change to `lastCollectionAt` in

`RecurringCollector.sol:1153-1183`, and the full open window is charged under the new rate in `RecurringCollector.sol:979-1003`. The following statement was provided by the `The Graph` team:

> This is intended behavior and is documented on RecurringCollector.update(). An update requires the payer's authorization (signature or stored offer) and only takes effect when the service provider submits it through the data service, so both parties consent to its application. A payer approving a rate increase does so knowingly. A provider facing a rate decrease can collect under the current terms before applying it. Maximum claim accounting, which contract payers use for escrow, already values pending updates from the last collection, consistent with this model. Introducing an effective timestamp would add significant complexity to the collector's state, the signed type, and the claim accounting, for no practical gain.

HAL-011 RISK ACCEPTED

Legacy delegate() lacks slippage protection, enabling sandwich attacks via addToDelegationPool()

Low · 3.1 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:L/Y:N

DESCRIPTION

The legacy `HorizonStaking.delegate(address serviceProvider, uint256 tokens)` entry point, kept for compatibility with the subgraph data service, calls the internal delegation path with the minimum share output hardcoded to zero, so it carries no slippage protection. `HorizonStaking.addToDelegationPool()` lets any address add tokens to an existing pool, increasing the pool's token balance without minting shares. Since shares are computed as the delegated tokens multiplied by pool shares divided by pool tokens net of thawing, a donation landing between a delegator's decision and their transaction reduces the shares they receive, and the difference accrues to existing share holders.

The economics constrain this considerably and the finding should say so. An attacker donating an amount to the pool recovers only their own ownership fraction of it, plus the same fraction of the victim's dilution. The attack is profitable only when the attacker already holds close to the entire pool and the incoming delegation is large relative to it, which is a narrow configuration. Deployment on a chain that does not expose a public transaction pool further limits the ability to order a donation immediately ahead of a specific delegation.

What remains is the absence of a protection the modern entry point provides, on a path that is still reachable, in a system where any address can move the share price. That is worth closing even where the present incentive to abuse it is weak, because the incentive changes with pool concentration and the missing check costs nothing.

RECOMMENDATION

Give the legacy entry point a slippage bound: compute a reasonable expected share output from current pool state and apply a configurable tolerance, or deprecate the entry point and direct callers to the version that accepts a minimum share output. Independently, consider restricting `HorizonStaking.addToDelegationPool()` to the service provider and authorized operators, since an open donation path into a pool whose price anyone can move has few legitimate uses beyond the recovery case described elsewhere in this report.

REMEDIATION COMMENT

RISK ACCEPTED: The legacy delegation path without a minimum-share setting was retained by the `The Graph` team. A zero minimum was verified in `HorizonStaking.sol:321-325`, while the pool price can

be changed by an unrestricted deposit in `HorizonStaking.sol:249-267`. The following statement was provided by the `The Graph` team:

> The legacy `delegate(address,uint256)` is kept for backwards compatibility and is explicitly documented as having no slippage protection. It must be preserved because vesting contracts (token lock wallets) can only call pre-approved function signatures and cannot use the new interface. Users who can call it should use `delegate(serviceProvider, verifier, tokens, minSharesOut)` for slippage protection.
>
> As the finding notes, a donation-based sandwich is unprofitable unless the attacker already holds nearly the entire pool. `MIN_DELEGATION` prevents near-empty pools, and Arbitrum does not expose a public mempool.
>
> Restricting `addToDelegationPool` to the service provider and its operators is not viable: it is the mechanism data services use to pay delegation fees and indexing rewards (for example, `GraphPayments` and `SubgraphService`), and those callers are neither.

HAL-012 SOLVED

Retroactive rate increase during pause violates the stated issuance upper bound invariant

Low · 3.1 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:L

DESCRIPTION

The `IssuanceAllocator` contract header declares an Issuance Upper Bound invariant stating that total issuance over any period never exceeds the sum of configured rates during that period. This invariant is a paper proof with no on-chain enforcement and is violated by a documented governance flow.

When the contract is paused, `lastDistributionBlock` freezes while blocks continue to advance. The two-argument overload of `IssuanceAllocator.setIssuancePerBlock()` accepts a `minDistributedBlock` at or below the frozen `lastDistributionBlock`, allowing the governor to write a new rate to storage without first distributing the elapsed window. When the contract is later unpaused and `IssuanceAllocator._distributePendingIssuance()` executes, it computes the budget as the new rate multiplied by the entire elapsed paused window. For a rate increase, the difference between the new rate and the old rate multiplied by the number of paused blocks is minted beyond what the rates actually in force during those blocks would have permitted.

This retroactive application is acknowledged in the function documentation with a governance caution note, and the single-argument overload is guarded against it, since it requires distribution to have reached the current block before applying. However, the header invariant makes a stronger claim than the code enforces: its proof chains the total minted for a period to the sum of per-block configured rates via the final rate multiplied by blocks, an inequality that is false whenever the final rate exceeds the rates in force during the window. The stated guarantee that total issuance never exceeds the sum of configured rates during the period, even when `issuancePerBlock` changes over time, is therefore violated by a documented governance flow, and no on-chain reconciliation enforces the claimed bound.

RECOMMENDATION

Either enforce the invariant on-chain or correct the documentation to match the code. On-chain enforcement could require that `lastDistributionBlock` has reached the current block before accepting a rate increase, forcing governance to first distribute the elapsed window at the old rate via `IssuanceAllocator.distributePendingIssuance()`. Alternatively, cap the distribution budget for any window to the sum of rates actually configured during each block in that window. If retroactive application is retained as intended behavior, the Issuance Upper Bound invariant and its closing guarantee in the contract header should be rewritten to state that the bound holds only when the rate is not increased over an undistributed window

REMEDIATION COMMENT

Addressed in: 1940caf24e833227b830d975e66604d74cacd116

SOLVED: The issuance invariant was qualified by the `The Graph` team. The corrected derivation and closing guarantee in `IssuanceAllocator.sol:61-113` now identify the exception for a rate increase over an undistributed window. The intended retroactive setter behavior remains documented in `IssuanceAllocator.sol:449-479`. The following statement was provided by the `The Graph` team:

> Retroactive application of a rate change made while paused is intended and documented on `setIssuancePerBlock(uint256,uint256)`: it requires an explicit governor action during a pause, and `distributePendingIssuance(blockNumber)` gives governance precise control over the block from which the new rate applies. We agree that the Issuance Upper Bound invariant in the contract header overstates the guarantee. We will update it to state that the bound holds with respect to the rate in effect at distribution time, and does not hold when governance intentionally increases the rate over an undistributed window.

HAL-013 SOLVED

Revoked collector retains just in time deposit capability and can extract the deposited funds

Low · 3.1 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:L/Y:N

DESCRIPTION

`RecurringAgreementManager._getAgreementProvider()` returns the cached provider for an already tracked agreement without rechecking `COLLECTOR_ROLE`. The role check gates only the first encounter with a new agreement, and the inline comment states this is intended: a deauthorized collector's agreements can still be reconciled, settled and force removed. A collector whose role has been revoked can therefore still call `RecurringAgreementManager.beforeCollection()` for any agreement it registered while authorized.

The `tokensToCollect` argument to `beforeCollection()` is supplied by the caller and bounded only by the manager's own token balance. The deposit logic computes the deficit as `tokensToCollect` minus the current escrow balance and deposits it when the deficit is below `GRAPH_TOKEN.balanceOf(address(this))`, so one call can move almost the entire balance of the manager into `escrowAccounts[manager][collector][provider]`.

Those funds are not merely misallocated. They are extractable by the same collector. `PaymentsEscrow.collect()` authorizes on the caller alone, reading `escrowAccounts[payer][msg.sender][receiver]`, and forwards to `GraphPayments.collect(paymentType, receiver, tokens, dataService, dataServiceCut, receiverDestination)`. Both `dataService` and `dataServiceCut` are chosen by the collector, and `GraphPayments` validates the cut only with `PPMMath.isValidPPM`, so a cut of 1000000 sends the whole amount, less the protocol cut, to an address the collector nominates. The deposit is the funding step of a direct theft rather than a liquidity inconvenience. `RecurringAgreementManager.forceRemoveAgreement()` followed by escrow thawing and withdrawal is a remedy only if the operator acts first, which it cannot, because the deposit and the withdrawal fit in one transaction.

The precondition is that an address holding `COLLECTOR_ROLE` is or becomes malicious. In the intended deployment that role belongs to `RecurringCollector`, a protocol contract, so this is a role management failure rather than a permissionless attack. It is exactly the failure that revoking the role is meant to contain, and revocation does not contain it.

RECOMMENDATION

Cap the just in time deposit at the agreement's `maxNextClaim` read from the collector, rather than trusting the caller supplied `tokensToCollect`, so a single call cannot move more than the agreement could legitimately claim. Independently, recheck `COLLECTOR_ROLE` in

`RecurringAgreementManager.beforeCollection()` for tracked agreements. Deposit is the one path on which a revoked collector gains new capability rather than merely settling an existing obligation; reconciliation and force removal can keep their current permissive behaviour.

REMEDIATION COMMENT

Addressed in: 1940caf24e833227b830d975e66604d74cacd116

SOLVED: The pause and forced-removal response was documented by the `The Graph` team in `RecurringAgreementManager.sol:87-96`. Limitation: A revoked collector with a tracked agreement can still trigger a deposit in `RecurringAgreementManager.sol:319-339` and `RecurringAgreementManager.sol:631-646` and draw escrow through `PaymentsEscrow.sol:163-190` before that response. The response time and deployed roles were not verified. The following statement was provided by the `The Graph` team:

> The precondition is a malicious `COLLECTOR_ROLE` holder, which in the intended deployment is the governance-controlled `RecurringCollector`. A malicious authorized collector already has this capability, and revocation is documented as non-retroactive for tracked agreements. Revoking the role is not, by itself, the containment mechanism for a compromised collector: reconciliation also trusts `getMaxNextClaim`, so the proposed `beforeCollection` check would not fully contain one. Containment is achieved by pausing the manager, which blocks the collection callbacks and reconciliation, followed by `forceRemoveAgreement`, which remains available during pause. We will update the escalation guidance in the contract documentation to say this explicitly.

HAL-014 RISK ACCEPTED

Payer-controlled eligibility callback can indefinitely withhold earned recurring fees

Low · 3.1 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:L/Y:N

DESCRIPTION

When a recurring collection agreement is accepted with the `CONDITION_ELIGIBILITY_CHECK` flag set, every call to `RecurringCollector._collect()` invokes the payer's `isEligible()` callback before releasing tokens. If the payer contract returns zero, collection reverts with `RecurringCollectorCollectionNotEligible`. There is no timeout, no provider-side override, and no fallback that allows the service provider to claim already-earned fees through an alternative path. Since the payer is an untrusted party that opted into this condition at acceptance time, a payer that persistently returns zero can strand all accrued fees in escrow indefinitely while the provider has no recourse within the protocol.

RECOMMENDATION

Introduce a timeout or provider-side pull mechanism so that a payer answering ineligible beyond a reasonable grace period cannot indefinitely block collection of fees that the provider has already earned. The eligibility gate should prevent premature collection, not serve as an indefinite veto over settled obligations.

REMEDIATION COMMENT

RISK ACCEPTED: The payer eligibility condition was retained by the `The Graph` team. An explicit ineligible callback result was verified to revert collection before payment in `RecurringCollector.sol:821-851`; no collector-level timeout was added. The following statement was provided by the `The Graph` team:

> `CONDITION_ELIGIBILITY_CHECK` is an opt-in agreement term, signed by the payer and accepted by the service provider. Its purpose is to let the payer make payment conditional on provider eligibility, so fees are only earned while the provider is eligible. The trust this places in the payer is documented on the flag in `RecurringCollector`. Only an explicit ineligible answer blocks collection; a reverting or malformed callback lets collection proceed. A collector-level timeout would let an ineligible provider collect after the grace period, which defeats the purpose of the condition. Liveness protections belong in the payer's eligibility source, and the protocol's `RecurringAgreementManager` provides them: the RewardsEligibilityOracle fails open after a timeout, and governance can clear the oracle in an emergency. Service providers that do not trust a payer's eligibility logic should not accept agreements with this condition.

HAL-015 RISK ACCEPTED

Authorized operator can redirect an indexer's payment destination

Low · 2.5 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:L/Y:N

DESCRIPTION

The payment destination of an indexer is written by two functions with inconsistent access control. `SubgraphService.setPaymentsDestination()` takes no address parameter and writes the destination only for `msg.sender`, so an indexer is the only party that can change where its own funds are sent. `SubgraphService.register()` writes the same storage through `_setPaymentsDestination(indexer, ...)`, but its only gate is the provision authorization check, which returns true for the indexer and for every operator the indexer has authorized on this verifier.

Because `SubgraphService.register()` does not require the indexer to be unregistered, it can be called against an already registered indexer at any time. An authorized operator can therefore re-register the indexer with the existing URL and geohash while supplying a different payment destination. Nothing changes off chain, since the URL and geohash are preserved.

Once the destination is overwritten, every payout routes to the new malicious destination. Query fees resolve the destination through `SubgraphService.collect()` when it encodes the collector data, indexing fees resolve it in `SubgraphService.collect()` when collecting the agreement, and the indexer share of indexing rewards is pushed to the destination inside `AllocationHandler.presentPOI()` during reward distribution. The operator can overwrite the destination and call `SubgraphService.collect()` in the same transaction, so although a payment destination event is emitted, detection does not prevent the loss.

This breaks the non extractive property of the operator role. The staking layer gives operators no path to move an indexer's principal to themselves, and this behavior creates one for all future revenue, including revenue backed by receipts signed before the operator was compromised.

RECOMMENDATION

Stop writing the payment destination from `SubgraphService.register()`. Restrict changes to the payment destination to the dedicated setter that is already scoped to `msg.sender`.

If the three field registration payload must be retained for ABI compatibility, only allow the destination to be written from `SubgraphService.register()` when the caller is the indexer itself, and otherwise ignore the supplied value.

REMEDIATION COMMENT

RISK ACCEPTED: Operator control of the registration payment destination was retained by the `The Graph` team. The caller-supplied destination is stored in `SubgraphService.sol:134-145` after provision authorization in `SubgraphService.sol:597-605`. The indexer's payment destination can still be changed at registration by an authorized operator. The following statement was provided by the `The Graph` team:

> This is intended. Operators are trusted parties authorized by the indexer to act on its behalf, and are often the indexer themselves operating through a hot key. Authorized operators already manage the indexer's allocations, POIs, fee collection and provision parameters, so setting the payments destination at registration is consistent with that trust model. An indexer should only authorize operators it trusts, and can revoke an operator and reset the destination with `setPaymentsDestination` at any time.

HAL-016 SOLVED

Provision parameter acceptance does not enforce the documented registration requirement

Low · 2.5 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

`SubgraphService.acceptProvisionPendingParameters()` documents that the indexer must be registered, but the function runs with the default requirement set, which performs only the pause check and the caller authorization check. The registration branch never executes, so provision parameters can be accepted for an indexer that was never registered.

No value is at risk. The function only validates and accepts staged provision parameters. This is a divergence between the specification and the code rather than an exploitable condition. Given that a strict registration requirement would compound the provision lockout described in the dispute period finding, the lenient behavior is arguably the correct one and the specification is the part that is wrong.

RECOMMENDATION

Resolve the divergence explicitly. Either add the registration requirement to `SubgraphService.acceptProvisionPendingParameters()` to match the documentation, or update the documentation to state that registration is not required.

REMEDIATION COMMENT

Addressed in: 1940caf24e833227b830d975e66604d74cacd116

SOLVED: The stale registration requirement was removed from the provision-acceptance NatSpec by the `The Graph` team. The corrected documentation and the lenient acceptance path were verified in `SubgraphService.sol:153-169`; registration still requires a valid provision in `SubgraphService.sol:589-605`. Deployment of the documentation change was not verified. The following statement was provided by the `The Graph` team:

> We agree the documentation is wrong and that the lenient behavior is correct. The registration requirement was intentionally removed to prevent an indexer lockout: `register` requires a valid provision, so an indexer whose provision parameters fall outside the accepted range must be able to accept corrected parameters before registering. We will remove the stale requirement from the NatSpec.

HAL-017 RISK ACCEPTED

Agreement term validation does not bound the per entity fee rate

Low · 2.5 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

The term validation for indexing agreements checks only that the base per second rate does not exceed the ceiling supplied by the recurring collector agreement. The amount actually requested at collection time in `IndexingAgreement.collect()` is the collection seconds multiplied by the base per second rate plus the per entity per second rate multiplied by the entity count. The per entity per second rate is never validated, and the entity count is supplied by the indexer at collection time.

As a result, an agreement can set a small or zero base rate to pass validation while carrying a large per entity per second rate, and the indexer can then drive the requested amount arbitrarily high by supplying a large entity count. The recurring collector truncates the actual payout to the agreement ceiling, so the payer is not overcharged, but the validation does not achieve what it appears to, and the indexer can always claim the full ceiling regardless of the agreed terms or the real entity count.

The requested amount computation can also overflow for a large entity count and revert, which is a self inflicted denial of service for the indexer only. The dispute path over the entity count is the only remaining control on inflated entity counts, and the dispute squatting finding removes that control, which makes this unpoliceable in combination.

RECOMMENDATION

Validate the full pricing surface against the agreement ceiling, including the per entity per second rate and a bound on the entity count, rather than only the base per second rate. Consider validating the maximum requestable amount per collection so the on chain terms match the effective ceiling enforced by the collector.

REMEDIATION COMMENT

RISK ACCEPTED: The existing agreement-term validation was retained by the `The Graph` team. Only the base rate is bounded in `IndexingAgreement.sol:784-793`, while the per-entity rate is stored in `IndexingAgreement.sol:634-644`. That term is not bounded at acceptance by the collector ceiling in `RecurringCollector.sol:1000-1003`. The following statement was provided by the `The Graph` team:

> The base rate check is a sanity check that the agreement terms fit within the RCA. The payer's binding protection is the RCA ceiling enforced by RecurringCollector, which caps every collection regardless of the requested amount, as documented on IndexingAgreement.collect. The per-entity

> rate cannot be meaningfully validated against the ceiling without an on-chain bound on entity count, which the protocol has no basis to set. The entity count is reported by the indexer alongside the POI, and misreporting is subject to indexing fee disputes, which remain effective (see our response to FIND-017). An overflow from an absurd entity count only reverts the indexer's own collection.

HAL-018 RISK ACCEPTED

Unbounded Stake Claims Traversal in collect() Critical Path Causes Gas DoS

Low · 2.5 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

Both `_collectQueryFees()` and `_collectIndexingFees()` in `SubgraphService` contract unconditionally call `_releaseStake(_indexer, 0)` before performing their respective collections. Passing `0` as the `numClaimsToRelease` parameter causes the LinkedList `traverse()` function to process ALL accumulated stake claims in one transaction (up to `MAX_ITEMS = 10,000` per the `LinkedList` library). Each `collect()` operation for query fees or indexing fees also adds a new stake claim via `_lockStake`. Since claims are only released when `releasableAt <= block.timestamp` (i.e., after the dispute period), an indexer that performs frequent small collections accumulates many un-expired stake claims in the linked list. When the dispute period expires for a large batch simultaneously, subsequent `collect()` calls must traverse and delete all accumulated expired claims, consuming O(n) gas per iteration. With `MAX_ITEMS = 10,000` and each iteration involving multiple SLOADs, SSTOREs, ABI encode/decode, and a ProvisionTracker release, the gas cost can easily exceed the block gas limit, permanently blocking the indexer from collecting any fees.

RECOMMENDATION

Use a small fixed batch size, e.g., `_releaseStake(_indexer, MAX_RELEASE_PER_COLLECT)` where `MAX_RELEASE_PER_COLLECT` is a constant like 10–50. This ensures O(1) gas for the release portion regardless of accumulated claims.

REMEDIATION COMMENT

RISK ACCEPTED: The unbounded release call in fee collection was retained by the `The Graph` team. Both collection paths pass zero to `_releaseStake` in `SubgraphService.sol:674-675` and `SubgraphService.sol:792-799`, and zero traverses all expired claims under `DataServiceFees.sol:61-80`. Bounded manual release remains available through `DataServiceFees.sol:30-32`. The following statement was provided by the `The Graph` team:

> Traversal stops at the first unexpired claim, so each collection only processes claims that expired since the previous one, and accumulating a gas-limit-sized batch requires thousands of collections by the indexer itself within a single dispute period. The condition is not permanent: `releaseStake(numClaimsToRelease)` exists for this case and is documented as the mechanism to use when releasing all claims at once would exceed the block gas limit. The indexer can release the backlog in batches and resume normal collection. A fixed per-collection cap would instead risk claims accumulating faster than they are released for high-frequency collectors. See also our response to FIND-047.

HAL-019 RISK ACCEPTED

Permissionless Accumulator Mutators Allow Caller to Bias Reward Distribution

Low · 2.5 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:L

DESCRIPTION

Four reward-accumulator mutators in `RewardsManager` contract are permissionless: `RewardsManager.updateAccRewardsPerSignal()`, `RewardsManager.onSubgraphSignalUpdate()`, `RewardsManager.onSubgraphAllocationUpdate()`, and `RewardsManager.beforeIssuanceAllocationChange()`. Each triggers a segment close that snapshots the current `accRewardsPerSignal` using the external `graphToken().balanceOf(curation)` value as the reward-per-signal denominator.

Because that denominator is a live external balance rather than an internally accounted total, an unprivileged caller can choose which block closes a multi-block accrual segment. Closing at a denominator trough yields a higher per-signal increment for that segment; closing at a peak yields a lower one. The cumulative `accRewardsPerSignal` therefore becomes path-dependent, where its final value depends on when segments were closed, not solely on the time-weighted average of the denominator over the period.

Standalone impact is limited by three factors. `RewardsManager.updateAccRewardsPerSignal()` is idempotent per block, so repeat calls within the same block have no additional effect. `L2Curation` internally calls `RewardsManager.onSubgraphSignalUpdate()` before transferring tokens during `L2Curation.mint()` and `L2Curation.burn()`, so a caller cannot front-run the Curation contract's own intra-transaction snapshot to exploit a known pending balance change. And the resulting per-signal increment is shared proportionally across all signal holders, so an attacker gains no individual enrichment from biasing it, where the effect is a distortion of protocol-wide accounting rather than extraction.

The finding becomes materially more significant in composition. Because any address can transfer GRT directly to the `L2Curation` contract to inflate `graphToken().balanceOf(curation)`, an actor can control both the denominator value and the timing of the segment close. Together these give a permission-less caller two coordinated levers over the global reward accumulator, which is why the accumulator denominator and the ungated mutators should be assessed as a single manipulable-denominator surface rather than as independent issues.

RECOMMENDATION

Derive the reward-per-signal denominator in `RewardsManager` from internally accounted curation pool token totals rather than `graphToken().balanceOf(curation)`, so the denominator cannot be moved

by an unsolicited transfer and the instant of segment closure becomes value-neutral. Alternatively, make the per-signal increment time-weighted so that the cumulative integral is independent of when segments are closed.

REMEDIATION COMMENT

RISK ACCEPTED: The permissionless checkpoint model was retained by `The Graph` team. The live curation balance is still used in `RewardsManager.sol:372-390`, and accumulator updates remain public in `RewardsManager.sol:497-509,581-639`. Repeat updates within one block are limited, but timing dependence remains. The following statement was provided by `The Graph` team:

> This shares the root cause of FIND-003: the rewards denominator is the curation contract's GRT balance, a known and long-standing design choice. As the finding notes, the standalone effect is not extractable. Updates are idempotent per block, curation mint and burn snapshot before changing the balance, and any bias is shared proportionally across all signal holders. Permissionless checkpointing is intentional, and restricting it would not remove timing dependence, since curation and allocation actions also close segments. We do not plan to change the accrual model.

HAL-020 RISK ACCEPTED

Proof of indexing validity is never verified on chain

Low · 2.5 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

`AllocationHandler.presentPOI()` accepts any bytes32 value as a proof of indexing and uses it only to distinguish the zero POI reclaim path from the normal reward path. The function checks staleness, maturity, and whether the subgraph is denied, but it never cryptographically or computationally verifies that the submitted POI corresponds to correct indexing work. When all four conditions pass, `RewardsManager.takeRewards()` mints GRT based solely on the allocation's size and duration, regardless of whether the POI is valid.

Correctness enforcement rests entirely on the optimistic dispute game in `DisputeManager`, where a fisherman can file a dispute against a fraudulent POI after the fact. This is the intended design, but it means that the security of indexing reward minting depends completely on the dispute mechanism functioning correctly and remaining accessible. Any defect that weakens the dispute path, such as the dispute identifier front-running and permanent immunization issue, removes the only check on fraudulent reward claims.

RECOMMENDATION

If the optimistic dispute model is retained, ensure the dispute path cannot be circumvented or permanently blocked for any POI tuple so the off-chain verification always has a functioning on-chain enforcement backstop. Document the trust assumption explicitly so downstream consumers understand that reward minting is not self-validating.

REMEDIATION COMMENT

RISK ACCEPTED: The optimistic proof-of-indexing design was retained by the `The Graph` team. Reward collection can be reached with a nonzero proof after the listed gates in `AllocationHandler.sol:335-364`, without an on-chain correctness check. Dispute creation remains available in `DisputeManager.sol:130-153`. The following statement was provided by the `The Graph` team:

> This is the intended design of the protocol. POIs cannot be verified on chain without re-executing indexing, so correctness is enforced optimistically through disputes and slashing, governed by the Arbitration Charter. The dispute path cannot be blocked for a given POI (see our response to FIND-017). The optimistic security model is documented in the protocol documentation and the Arbitration Charter.

HAL-021 SOLVED

Revoked signer addresses are permanently unusable protocol-wide

Low · 2.5 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

`Authorizable.revokeAuthorizedSigner()` marks an authorization as revoked but leaves the signer's recorded authorizer in place. Because `Authorizable.authorizeSigner()` only proceeds when the recorded authorizer is the zero address, a revoked signer can never be authorized again. The authorization mapping is keyed on the signer address alone rather than on the authorizer and signer pair, so the address is consumed globally and cannot be reclaimed by the original authorizer or by any other party.

Routine key rotation, a mistake during initial binding, or revoking a compromised signer all result in an address that is permanently dead across the protocol with no rescue path. The contract documents that signers cannot be shared across different authorizers, but the inability to rebind after revocation is not covered by that statement.

RECOMMENDATION

Clear the recorded authorizer when an authorization is revoked so the address can be authorized again in the future. The proof verification already binds the caller, chain identifier, and verifying contract, so reauthorization would still require a fresh proof and the existing protection against a third party claiming the address is preserved. A dedicated rebind function restricted to the original authorizer would achieve the same outcome.

REMEDIATION COMMENT

Addressed in: 1940caf24e833227b830d975e66604d74cacd116

SOLVED: Permanent signer revocation was documented by the `The Graph` team in `Authorizable.sol:12-32`. Limitation: The runtime behavior in `Authorizable.sol:76-115` is unchanged; a revoked signer address cannot be authorized again, and rotation requires a new address. The following statement was provided by the `The Graph` team:

> Permanent revocation is intentional. Revocation is commonly used for compromised keys, and allowing a revoked address to be rebound would let anyone holding the key, including an attacker, produce a fresh proof and revive it. Signer keys are cheap to generate, so key rotation is done by authorizing a new signer. We will extend the contract documentation to state explicitly that revocation is permanent.

HAL-022 RISK ACCEPTED

Self authorization override disarms the signer access modifier on three mutating functions

Low · 2.5 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

`RecurringCollector._isAuthorized()` returns true whenever the signer argument is the collector's own address, an override written so the collector can satisfy authorization checks that data services perform on its behalf. The base function is virtual and `Authorizable._requireAuthorized()` dispatches through it, so the override also redefines the `onlyAuthorized` guard on `Authorizable.thawSigner()`, `Authorizable.cancelThawSigner()` and `Authorizable.revokeAuthorizedSigner()`. The modifier's parameter is the signer, not the caller, so passing the collector's own address satisfies the check before the caller is examined at all. Any account can therefore write to the collector's authorization record and emit signer events attributed to itself.

The record is empty and stays empty, since `Authorizable.authorizeSigner()` needs an ECDSA proof recovering to a contract address, so nothing genuine is overwritten. The reason to fix it is ordering. The self trust branch runs before the base implementation, which is where the revoked flag is read, so revoking the collector cannot switch off its own self trust today. Reorder those branches, or move the revoked check ahead of the special case in a later refactor, and that same unprivileged call permanently breaks `IndexingAgreement.cancelByPayer()` for every indexing agreement.

RECOMMENDATION

Have the modifier assert that the caller is the recorded authorizer for the signer, rather than routing through a predicate subclasses can override. Alternatively confine the override to the read path it was written for and reject the collector's own address explicitly in the three mutating functions.

REMEDIATION COMMENT

RISK ACCEPTED: The self-authorization behavior was retained by the `The Graph` team, and a warning was added to `RecurringCollector.sol:1378-1383`. The guard still calls the overridable predicate in `Authorizable.sol:63-65` and `Authorizable.sol:146-148`, so the three signer mutators remain callable by any address for the collector's own signer record. The following statement was provided by the `The Graph` team:

> Confirmed, with no impact in the current code: the collector's own authorization record can never be authorized, and the self-trust branch does not read it, so these calls only emit events and write to an unused record. We will document on the override that it also governs the ònlyAuthorized`guard

> and that the self-trust branch must not depend on the stored authorization state. Any future refactor of this logic will decouple the modifier from the overridable predicate.

HAL-023 RISK ACCEPTED

Signer revocation thawing period accepts any value while the documented bound is enforced only on the escrow side

Low · 2.5 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

The `Authorizable` constructor assigns `REVOKE_AUTHORIZATION_THAWING_PERIOD` with no validation, `RecurringCollector` forwards its own argument unchecked, and neither contract exposes a setter, so a wrong value can only be corrected by redeploying.

The bound already exists, in documentation and in the wrong contract. `PaymentsEscrow.MAX_WAIT_PERIOD` is ninety days and its comment states that this is the maximum thawing period for both escrow withdrawal and collector revocation. `PaymentsEscrow` enforces it on withdrawal in its constructor. Nothing enforces it on revocation.

Any value in the range of years leaves revocation nominally available and useless, and revocation is the only working remedy for a compromised signer. Large enough and the failure is total: `Authorizable.thawSigner()` computes the thaw end with checked arithmetic and reverts on overflow, and since `Authorizable.revokeAuthorizedSigner()` requires a thaw end that only `Authorizable.thawSigner()` can set, revocation becomes permanently unreachable.

RECOMMENDATION

Bound the revocation thawing period in the `Authorizable` constructor, following the precedent already set for the withdrawal period in `PaymentsEscrow`, so a wrong value fails at deployment rather than when a signer needs revoking.

REMEDIATION COMMENT

RISK ACCEPTED: The unrestricted constructor parameter was retained by the `The Graph` team. The revocation thawing period was verified to be assigned without a bound in `Authorizable.sol:68-74`; the limit in `PaymentsEscrow.sol:56-62` applies only to escrow withdrawal. The following statement was provided by the `The Graph` team:

> `REVOKE_AUTHORIZATION_THAWING_PERIOD` is an immutable deployment parameter set by the protocol deployer and publicly readable. The deployed values are 30 days for GraphTallyCollector and 8 hours for RecurringCollector, both within the documented bound. We will consider adding a constructor bound the next time these contracts are redeployed.

HAL-024 RISK ACCEPTED

Undelegation derives the unbonding deadline from a provision it never requires to exist

Low · 2.5 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

`HorizonStaking._delegate()` requires the provision to exist. `HorizonStaking._undelegate()` does not, yet reads the thawing period from it. With no provision the struct is entirely zero, the deadline resolves to the current block timestamp, and `HorizonStaking._fulfillThawRequest()` compares with a strict greater than, so the request matures in the block that created it. `HorizonStakingBase` inherits multicall, so `HorizonStaking.undelegate()` and `HorizonStaking.withdrawDelegated()` batch into one transaction.

Only legacy delegation reaches this state, since `HorizonStakingBase._getDelegationPool()` routes the subgraph data service to a mapping that predates the provision model. Every other route to holding shares passes the check in `HorizonStaking._delegate()`.

No value is at risk today. Slashing needs a live allocation in `SubgraphService`, an allocation needs a provision, and a provision is never deleted once created, so the two conditions exclude each other. The delegation also secures nothing, because `HorizonStakingBase.getTokensAvailable()` caps delegated capacity at the provider's available tokens times the delegation ratio, which is zero when the provision holds none. What remains is an exit path that resolves a missing struct to a zero deadline and is safe only because of an invariant held in a different package.

RECOMMENDATION

Require the provision to exist in `HorizonStaking._undelegate()`, mirroring the entry side. Where a legacy delegator legitimately has none, apply an explicit governed fallback unbonding period rather than defaulting silently to zero.

REMEDIATION COMMENT

RISK ACCEPTED: The missing-provision undelegation behavior was retained by the `The Graph` team. The unbonding deadline was verified to use the provision's thawing period without an existence check in `HorizonStaking.sol:797-813`, so a missing provision supplies a zero period. The following statement was provided by the `The Graph` team:

> This is intended. The delegation unbonding period is the provision's thawing period because delegated stake is only exposed to slashing through that provision. Without a provision there is no delegated capacity, no allocation and nothing to slash, so a zero unbonding period is correct.

> Requiring a provision on undelegation would trap legacy delegators of indexers that never provisioned to SubgraphService. Provisions are never deleted in HorizonStaking, so once a provision exists its thawing period always applies.

HAL-025 ACKNOWLEDGED

Flat curve lets signal be minted and burned around a query-fee collect() to skim injected fees

Informational · 1.7 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

`L2Curation` uses a flat bonding curve (100% reserve ratio), so signal redemption value is directly proportional to pool reserves: `signalToTokens()` returns `curationPool.tokens * signalIn / curationPoolSignal`. The `collect()` function adds query-fee tokens to a pool's reserve without minting any new signal, which instantaneously raises the redemption value of all existing signal.
An actor holding or acquiring signal immediately before a `collect()` and redeeming immediately after captures a pro-rata share of the injected fees. Framed as an ordering play: `mint()` before the injection acquires signal at the pre-injection ratio, and `burn()` after the injection redeems at the higher ratio. Because curation tax is charged only on `mint()` `burn()` is untaxed), the actor's only cost is the tax on the entry, and the play is profitable whenever the fee injection as a fraction of pool size exceeds the curation tax rate.
Exploitability is bounded by several factors that should be stated plainly:

- `collect()` is not permissionless. It is gated to the subgraph service `require(msg.sender == subgraphService, "Caller must be the subgraph service")`), so an attacker cannot insert or trigger the injection themselves; they can only react to a `collect()` initiated by the authorized caller.
- Capturing the injection requires ordering `mint()` and `burn()` around a specific `collect()`. On the L2 deployment the sequencer does not expose a public mempool for general-case sandwiching, which removes the standard front-run/back-run path and leaves only cases where the actor can otherwise anticipate injection timing.
- Per `collect()` fee amounts are typically small relative to pool reserves, so the tax barrier is binding in most realistic cases and the play is unprofitable.

The flat curve distributes an untaxed, signal-free reserve increase to whoever holds signal at that instant, so signal held only transiently around an injection earns the same as long-term curation signal. This rewards timing over genuine curation and lets a well-positioned holder dilute the injected fees away from persistent curators.

RECOMMENDATION

Consider decoupling redemption value from raw reserve injections so that a `collect()` cannot be captured by transiently-held signal; for example, stream collected fees into redeemable value over a period rather than applying them atomically, or account collected fees separately from the signal-

redeemable reserve. At minimum, document that `collect()` produces an atomic value step on a flat curve and that fee amounts per call should stay small relative to pool reserves so the curation tax remains a binding cost against timing plays.

REMEDIATION COMMENT

ACKNOWLEDGED: The atomic fee collection model was retained by `The Graph` team. Collection is restricted to the subgraph service in `L2Curation.sol:173-185`, where fees are added to the pool reserve without new signal. Redemption is then priced against that reserve in `L2Curation.sol:431-437`, so the value step at collection remains. The following statement was provided by `The Graph` team:

> This is a known property of curation: query fees accrue to signal holders at the time of collection, as they always have in the curation contracts. The curation tax on minting makes the play unprofitable unless a single collection exceeds the tax rate as a fraction of the pool. Collection timing is controlled by the indexer and cannot be observed in advance on Arbitrum. Streaming fees into the pool would be a redesign of curation economics, which we do not plan as part of this scope.

HAL-026 ACKNOWLEDGED

Off-by-one in RecurringAgreementManager.beforeCollection deposit condition causes JIT top-up failure when contract balance exactly equals deficit

Informational · 1.7 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

The `RecurringAgreementManager.beforeCollection` callback uses a strict less-than (`<`) comparison when checking if the contract has enough tokens to cover the escrow deficit. The condition `if (deficit < GRAPH_TOKEN.balanceOf(address(this)))` fails to deposit when the contract's token balance exactly equals the required deficit, leaving the escrow underfunded. The correct check should be `<=` to allow depositing when the contract balance exactly covers the needed amount.

RECOMMENDATION

Change the strict less-than comparison to less-than-or-equal in the deposit condition.

REMEDIATION COMMENT

ACKNOWLEDGED: The exact-balance top-up behavior was retained by the `The Graph` team. The strict comparison in `RecurringAgreementManager.sol:328-339` was verified to skip the deposit when the contract balance equals the escrow deficit. The following statement was provided by the `The Graph` team:

> Confirmed. The impact is limited to the case where the manager's balance exactly equals the deficit, and the just-in-time top-up is best-effort, so the affected collection can be retried. We will address it if RecurringAgreementManager is modified for other reasons.

HAL-027 ACKNOWLEDGED

GraphDirectory Immutable Protocol Addresses Cannot Be Updated Post-Deployment, Creating Stale Address Risk on Protocol Contract Migration

Informational · 1.7 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

The `GraphDirectory` contract stores 9 critical protocol contract addresses (`GRAPH_TOKEN`, `GRAPH_STAKING`, `GRAPH_PAYMENTS`, `GRAPH_PAYMENTS_ESCROW`, `GRAPH_CONTROLLER`, `GRAPH_EPOCH_MANAGER`, `GRAPH_REWARDS_MANAGER`, `GRAPH_TOKEN_GATEWAY`, `GRAPH_PROXY_ADMIN`) as `private immutable` variables. These are resolved from the Controller registry in the implementation constructor and baked into the bytecode. Because immutables are not stored in proxy storage, they cannot be updated without deploying a new implementation contract. If any of these 9 referenced proxy contracts ever migrates to a new address (i.e., the Controller's registered proxy for that contract changes), ALL contracts that inherit `GraphDirectory` with the old address would silently continue directing calls to the stale/obsolete address. There is no upgrade mechanism or fallback that allows dynamic address resolution after the initial constructor call.

RECOMMENDATION

Modify `GraphDirectory` contract to cache addresses in immutables but also provide an override mechanism (e.g., via a storage variable read when the immutable is zero, or a re-read from `GRAPH_CONTROLLER` at runtime). Instead of immutables, use a single `GRAPH_CONTROLLER` immutable and resolve all other addresses dynamically via `GRAPH_CONTROLLER.getContractProxy(keccak256("PaymentsEscrow"))` on each call. This adds one additional SLOAD per call (or STATICCALL for external resolution) but allows transparent updates when proxy addresses change.

REMEDIATION COMMENT

ACKNOWLEDGED: Immutable protocol-address caching was retained by the `The Graph` team. The addresses were verified to be set in the implementation constructor in `GraphDirectory.sol:96-107` and returned without a Controller re-read in `GraphDirectory.sol:122-191`; a changed proxy address requires replacement of dependent implementations. The following statement was provided by the `The Graph` team:

> This is an intentional gas optimization, documented in the GraphDirectory`contract. The cached addresses are proxy addresses, which remain stable across contract upgrades. Replacing a proxy would be a planned governance migration, and redeploying the dependent implementations so they re-read the Controller is part of that procedure.

HAL-028 ACKNOWLEDGED

Integer Division Truncation in createQueryDisputeConflict Causes Permanent Token Loss of 1 Wei

Informational · 1.7 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:N/D:L/Y:N

DESCRIPTION

In `DisputeManager.createQueryDisputeConflict()`, the function pulls `disputeDeposit` tokens from the fisherman and then creates two disputes, each assigned `disputeDeposit / 2` as their deposit. Since `disputeDeposit / 2` is integer division in Solidity, when `disputeDeposit` is an odd number, `dId1.deposit + dId2.deposit = 2 * (disputeDeposit / 2) = disputeDeposit - 1`. The fisherman pays `disputeDeposit` tokens but only `disputeDeposit - 1` are tracked in the two dispute deposits. The 1 token difference is neither refunded nor burned, as it becomes permanently stuck in the contract.

RECOMMENDATION

Track the total across both deposits and ensure nothing is lost. Alternatively, use `Math.ceilDiv()` for one of the disputes to ensure the sum equals `disputeDeposit()`.

REMEDIATION COMMENT

ACKNOWLEDGED: The odd-deposit division was retained by the `The Graph` team. The full deposit is pulled, while two floor-divided amounts are stored in `DisputeManager.sol:195-210`. One base unit remains unassigned when the deposit is odd. The following statement was provided by the `The Graph` team:

> Confirmed. The loss is at most 1 wei per conflicting-attestation dispute, and only when `disputeDeposit` is odd. The deployed value is 10,000 GRT, which is even, and the deposit is set by governance in whole GRT amounts. We will address it if `DisputeManager` is modified for other reasons.

HAL-029 ACKNOWLEDGED

Unbounded entities Parameter Causes Overflow and DoS in IndexingAgreement._tokensToCollect

Informational · 1.7 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

The `_tokensToCollect()` function in `IndexingAgreement` contract computes expected fees as:

```sol
return _collectionSeconds * (termsV1.tokensPerSecond + termsV1.tokensPerEntityPerSecond * _entities);
```

The `_entities` parameter is supplied by the indexer at collection time as a `uint256` and is completely uncapped. The terms validation `_validateTermsAgainstRCA()` only checks `tokensPerSecond <= maxOngoingTokensPerSecond` but does not validate or bound `tokensPerEntityPerSecond`. Because both `tokensPerEntityPerSecond` and `_entities` are unbounded uint256 values, their product can overflow. In Solidity 0.8+, this triggers an automatic revert, preventing any collection from completing.

Scenario:

- Agreement with `tokensPerEntityPerSecond = 1e18` (1 GRT per entity per second - reasonable value), `tokensPerSecond = 0`.
- Self-DoS scenario (accidental): a legitimate indexer that submits `entities = 1e60` (a very large subgraph) would overflow if `tokensPerEntityPerSecond >= 1e18`. Given GRT has 18 decimals, this is entirely plausible.
  The indexer loses the fee collection for that time window since `lastCollectionAt` is only updated inside the successful `collect()` path. The payer's escrow is also not released. The indexer must cancel and recreate the agreement to recover.

RECOMMENDATION

Add an upper bound check on `entities` in `_tokensToCollect` or at the caller level, or enforce a bound on `tokensPerEntityPerSecond` in `_validateTermsAgainstRCA()` to prevent overflow, or use `Math.mulDiv` from OpenZeppelin for overflow-safe computation.

REMEDIATION COMMENT

ACKNOWLEDGED: The unbounded entity multiplication was retained by the `The Graph` team. The collection amount is computed in `IndexingAgreement.sol:719-727` before the collector call in `IndexingAgreement.sol:559-577`. A sufficiently large entity count still reverts that collection. The following statement was provided by the `The Graph` team:

> This duplicates the overflow point in FIND-023. Overflow requires an entity count many orders of magnitude beyond any real subgraph, and it only reverts the indexer's own call. Because the revert leaves state unchanged, the collection window is not lost: the indexer resubmits with the correct entity count and collects the same period. No cancellation or re-creation of the agreement is needed.

HAL-030 ACKNOWLEDGED

Issuance accrues by block.number whose semantics are chain-dependent

• Informational · 1.7 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:N/D:L/Y:N

DESCRIPTION

`IssuanceAllocator` contract accrues all token issuance strictly by `block.number` deltas. `IssuanceAllocator._advanceSelfMintingBlock()` computes the self-minting offset as `totalSelfMintingRate` multiplied by the elapsed block delta, and `IssuanceAllocator._distributePendingIssuance()` computes the total budget as `issuancePerBlock` multiplied by the delta between `toBlockNumber` and `lastDistributionBlock`. All downstream distribution in `IssuanceAllocator._distributePendingWithFullRate()` and `IssuanceAllocator._distributePendingProportionally()` derives mint amounts from that same block delta.

On Arbitrum, `block.number` returns the L1 block number rather than the L2 transaction sequence number. This value is flat across L2 transactions within the same L1 block and then jumps by a batch of L1 blocks. The `issuancePerBlock` rate is therefore semantically bound to L1 block cadence rather than L2 execution cadence. If the rate is calibrated under an L1 mental model but deployed on Arbitrum, the actual minted GRT per unit of wall-clock time diverges from the intended inflation schedule, distorting protocol-wide token supply.

RECOMMENDATION

Accrue issuance on a monotonic, chain-appropriate time base such as `block.timestamp` or `ArbSys.arbBlockNumber()` on Arbitrum. If `block.number` is retained, pin the calibrated cadence with an on-chain assertion and document the `issuancePerBlock` semantics against the confirmed deployment chain so that governance sets the rate to match the actual block production schedule.

REMEDIATION COMMENT

ACKNOWLEDGED: The block-based issuance convention was retained by `The Graph` team. Self-minting and distribution amounts are still calculated from `block.number` deltas in `IssuanceAllocator.sol:380-392` and `IssuanceAllocator.sol:460-476`. The stated live rate calibration was not independently read. The following statement was provided by `The Graph` team:

> Accruing on `block.number`, which on Arbitrum is the L1 block number, is intentional and consistent with RewardsManager and EpochManager, which have used L1 block cadence since the protocol moved to Arbitrum. The deployed `issuancePerBlock` is calibrated against L1 blocks, and governance sets it with that convention.

HAL-031 ACKNOWLEDGED

EIP-712 domain separator is cached at initialization and not recomputed on chain ID change

● Informational · 1.7 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

`AttestationManager` is an abstract helper inherited by `DisputeManager`, which is the concrete contract where the behaviour is observable. During `DisputeManager.initialize()`, the call to `AttestationManager.__AttestationManager_init()` computes the domain separator once and stores it, embedding the chain identifier at that moment. `AttestationManager._encodeReceipt()` reads only the cached value and never recomputes it.

If the chain identifier changes after deployment, for example on a forked network that adopts a new identifier, the cached separator continues to encode the original one. Verification then runs against a domain that no longer matches the domain a compliant signer on that chain constructs. Two concrete consequences follow. `DisputeManager.createQueryDispute()` and `DisputeManager.createQueryDisputeConflict()` recover an unexpected address rather than the allocation identifier that signed the attestation, so legitimate disputes over correctly signed attestations fail. And `DisputeManager.encodeReceipt()` returns a hash inconsistent with what indexer signing software produces, so the two sides of the attestation flow disagree about what is being signed.

PROOF OF CONCEPT

Chain Fork Replay Attack:

Prerequisites: Ethereum/Arbitrum hard fork occurs while DisputeManager is deployed. Post-fork, two chains coexist with (temporarily) the same chain ID.

Attack Steps:

1. Before the fork, Indexer I signs an attestation A for subgraph deployment D, receipt R = (requestCID, responseCID, D). The signature is computed against the `_domainSeparator` which encodes `chainId = X` and `verifyingContract = DisputeManager_address`.
2. Fork occurs. Chain α retains chain ID X. Chain β eventually adopts chain ID Y.
3. During the brief window where both chains share chain ID X (before β updates), a fisherman on chain β can submit the attestation A to create a query dispute. The cached `_domainSeparator` was computed with chainId = X, which is still valid on β at this moment.
4. The indexer gets slashed on chain β for an attestation that was originally valid on chain α.

Cross-Contract Same-Address Replay (deterministic deployment):

1. Old `contracts/disputes/DisputeManager` and new `subgraph-service/contracts/DisputeManager` share identical DOMAIN_SALT, DOMAIN_NAME_HASH, DOMAIN_VERSION_HASH, RECEIPT_TYPE_HASH.
2. If deployed at the same address on different networks (same chain ID), the domain separator for both would be identical, allowing cross-contract attestation replay.

RECOMMENDATION

Recompute the domain separator when the current chain ID differs from the value captured at initialization, following the pattern used by OpenZeppelin's EIP712 implementation. This preserves the existing domain constants and produces an identical separator under normal operation, changing behavior only after a chain ID change. Optionally expose `eip712Domain()` per EIP-5267 so that wallets and tooling can display the domain parameters being signed over.

REMEDIATION COMMENT

ACKNOWLEDGED: The initialization-time attestation domain was retained by the `The Graph` team. The separator is set from `block.chainid` in `AttestationManager.sol:52-63` and reused for signer recovery in `AttestationManager.sol:70-108`. It is not recalculated after a chain ID change. The following statement was provided by the `The Graph` team:

> The cached domain separator follows the scheme used by the DisputeManager since launch, which indexer software relies on. A chain ID change would be an exceptional event requiring a coordinated protocol response, and query disputes are resolved by the arbitrator, who would not slash for an attestation replayed across forks. The replay scenarios described are not affected by the recommendation. During a shared-chain-ID window the recomputed separator would be identical. The cross-contract scenario requires two contracts at the same address on the same chain ID, which cannot occur, since the chain ID and verifying contract are part of the domain.

HAL-032 ACKNOWLEDGED

Missing __gap Storage Arrays in SubgraphServiceV1Storage and SubgraphServiceV2Storage Creates Storage Collision Risk on Future Upgrades

• Informational · 1.7 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

`SubgraphServiceV1Storage` and `SubgraphServiceV2Storage`, which form the application-layer storage backbone for the SubgraphService upgradeable proxy, do not include `__gap` storage arrays for upgrade safety. All other storage contracts in the same inheritance chain `DataServiceV1Storage`, `DataServiceFeesV1Storage`, `DataServicePausableUpgradeable`, `AllocationManagerV1Storage`, `AttestationManagerV1Storage`) correctly include `uint256[50] private __gap` arrays. The absence of `__gap` in the SubgraphService-specific storage contracts makes it impossible to safely add new state variables to these contracts in any future upgrade without creating a new versioned contract and risks that a developer mistake, adding a variable directly to V1Storage or V2Storage rather than creating a V3Storage, would corrupt the storage layout of all contracts below them in the inheritance chain, silently overwriting critical variables like `allocationProvisionTracker`, `_allocations`, `maxPOIStaleness`, and the `AllocationManagerV1Storage.__gap` itself.

RECOMMENDATION

Add `uint256[50] private __gap;` (or an appropriately sized gap) to both `SubgraphServiceV1Storage` and `SubgraphServiceV2Storage`, consistent with the pattern used by all other storage contracts in the same inheritance chain. Note: The gap size should account for existing variables (since `__gap` fills the remainder of the reserved space). A 50-slot reserve per storage contract is the established pattern in this codebase. Also add `__gap` to `DisputeManagerV1Storage` which has the same issue.

REMEDIATION COMMENT

ACKNOWLEDGED: The storage layouts without gaps were retained by the `The Graph` team. `SubgraphServiceV2Storage` inherits `SubgraphServiceV1Storage` and adds a field in `SubgraphServiceStorage.sol:16-41`, so V1 is a base contract in the current layout. Future changes to these layouts still require upgrade-safe storage placement. The following statement was provided by the `The Graph` team:

> These are the most-derived storage contracts in their inheritance chains, so no storage follows them and appending variables does not collide with anything. Storage gaps are needed only in base contracts that are inherited by others, which is where the codebase uses them. Future state is added through new versioned storage contracts, following the existing V1 to V2 pattern in

> SubgraphService.` A gap would not protect against a variable inserted into the middle of an existing layout.

HAL-033 ACKNOWLEDGED

Missing __gap Storage Array in HorizonStakingV1Storage Risks Storage Collision on Future Upgrades to the Protocol's Central Staking Contract

Informational · 1.7 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

The `HorizonStakingV1Storage` contract, which is the sole storage backbone of the `HorizonStaking` upgradeable proxy, does not include an OpenZeppelin-style `__gap` storage array. This is the most critical storage contract in The Graph protocol, holding all provision tokens, delegation pools, thaw requests, and operator authorizations. Without a `__gap`, the storage layout has no reserved slots between `HorizonStakingV1Storage`'s variables and any future storage contract appended during an upgrade. A developer who inadvertently adds a new state variable directly to `HorizonStakingV1Storage` (instead of creating a new `HorizonStakingV2Storage` appended at the end of the inheritance chain) would corrupt every mapping and variable following the insertion point: `_provisions`, `_delegationPools`, `_operatorAuth`, `_thawRequests`, and `_allowedLockedVerifiers`, all of which track locked GRT across the protocol.

RECOMMENDATION

Add a `__gap` array to `HorizonStakingV1Storage`, sized appropriately to bring the total reserved space to a round number (e.g., 50 slots from the start of the Horizon-specific section, or simply appending sufficient reserved slots at the end). Additionally, create `HorizonStakingV2Storage` for any future additions, when new state variables are required, create `HorizonStakingV2Storage extends HorizonStakingV1Storage` and have `HorizonStaking` inherit from it, following the same pattern used in SubgraphService's V1-to-V2 transition.

REMEDIATION COMMENT

ACKNOWLEDGED: The current staking storage layout without a trailing gap was retained by the `The Graph` team. `HorizonStakingV1Storage` was verified to end after `_allowedLockedVerifiers` in `HorizonStakingStorage.sol:179-186`; any later extension must preserve the established layout. The following statement was provided by the `The Graph` team:

> HorizonStakingV1Storage is the last storage-bearing contract in the HorizonStaking inheritance chain, and the remaining bases declare no state, so no storage follows it. Its layout continues the legacy Staking storage behind the existing proxy. Future state will be added through a new versioned storage contract appended to the chain, as the finding suggests. A gap would not protect against a variable inserted into the middle of the existing layout.

HAL-034 ACKNOWLEDGED

A provision can be created with no unbonding period at all

Informational · 1.7 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

`HorizonStaking._createProvision()` checks the thawing period against the governed maximum but applies no lower bound and accepts zero. Both `HorizonStaking._thaw()` and `HorizonStaking._undelegate()` derive their deadlines from that field, so a zero period gives the service provider and every delegator an unbonding period of zero.

A lower bound is enforced today, but one layer up and per data service. `ProvisionManager` validates the period against a configured range before a provider may register or operate, so containment depends on every data service being configured with a nonzero minimum. `HorizonStaking` itself will hold the provision, accept delegation into it, and release both stake and delegation with no delay. `HorizonStaking.setProvisionParameters()` has the same gap, with smaller exposure, since it only stages a value the verifier must accept and `ProvisionManager` revalidates at that point.

RECOMMENDATION

Enforce a governed nonzero minimum thawing period in `HorizonStaking._createProvision()` and `HorizonStaking.setProvisionParameters()` alongside the existing maximum.

REMEDIATION COMMENT

ACKNOWLEDGED: The generic staking policy that permits a zero thawing period was retained by the `The Graph` team. Only an upper bound was verified in `HorizonStaking.sol:587-620`, while `SubgraphService.sol:568-576` applies a service-specific dispute-period requirement. The following statement was provided by the `The Graph` team:

> This is intentional. HorizonStaking is a generic staking primitive, and each data service defines the economic security it requires, including the thawing period, which it enforces through ProvisionManager before a provider can operate. SubgraphService requires the thawing period to match the dispute period. A protocol-wide minimum would constrain data services that do not need delayed unbonding. A provision's thawing period is visible on chain to delegators choosing where to delegate.

HAL-035 ACKNOWLEDGED

Offer cancellation is recorded under the caller while authorization is checked under the recovered signer

Informational · 1.7 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

`RecurringCollector.cancel()` writes the cancelled offer under the message sender and the terms hash. `RecurringCollector._requireAuthorization()` reads that mapping under the address recovered from the signature. The two match only when the payer signs with its own key, which is the arrangement the authorization mechanism exists to avoid.

No other scope covers the gap. The pending scope returns early when the stored payer is the zero address, which is the state of any offer that exists only as a signature, and where a stored offer does exist, deleting it disables only the unsigned branch, since `RecurringCollector.accept()` rewrites it once the signature verifies. The active scope needs the agreement accepted already. Signer revocation works but only after the full thawing period in `Authorizable`, with no expedited path.

RECOMMENDATION

Check both the signer keyed and the payer keyed entry in `RecurringCollector._requireAuthorization()`. The payer keyed entry is already written by the existing cancellation path, so this needs no new storage and no new entry point.

REMEDIATION COMMENT

ACKNOWLEDGED: Signer-keyed cancellation of signed offers was retained by the `The Graph` team. Cancellation was verified to be stored under the caller in `RecurringCollector.sol:551-565` and checked under the recovered signer in `RecurringCollector.sol:1106-1125`; payer-keyed cancellation still does not block a separate signer key. The following statement was provided by the `The Graph` team:

> Signed-scope cancellation is intentionally self-authenticating, as documented in RecurringCollector.cancel: the signer that produced an offer cancels it by calling cancel itself, which any authorized signer key can do. For a compromised signer, per-offer cancellation is not an effective remedy, since the holder of the key can sign new offers, so revocation is the appropriate mechanism. We will consider also honoring payer-keyed cancellations in a future RecurringCollector upgrade.

HAL-036 ACKNOWLEDGED

DisputeManager Arbitrator Can Be Instantly Replaced With No Timelock, Enabling Single-Transaction Governance Takeover of All Dispute Resolution

Informational · 1.5 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:H/I:N/D:N/Y:N

DESCRIPTION

The `DisputeManager.setArbitrator()` function allows the owner to immediately replace the arbitrator address without any timelock or delay. The arbitrator is the sole entity authorized to resolve disputes; calling `acceptDispute()`, `rejectDispute()`, and `drawDispute()`. A malicious or compromised owner can instantly replace a legitimate arbitrator with an attacker-controlled address. This creates a single-transaction governance takeover path for the entire dispute resolution system: the new malicious arbitrator can immediately accept all pending disputes (slashing indexers), reject all disputes (destroying fisherman incentives), or draw disputes (returning deposits while misbehaving indexers escape punishment). There is no pending/timelock mechanism, no multi-step confirmation, and no notification period for active fishermen or indexers.

RECOMMENDATION

Implement a two-step arbitrator transfer with a time delay.

REMEDIATION COMMENT

ACKNOWLEDGED: The immediate arbitrator setter was retained by the `The Graph` team. The arbitrator can be changed by the owner through `DisputeManager.sol:282-284`, and the new address is stored without delay in `DisputeManager.sol:648-652`. No two-step transfer is required. The following statement was provided by the `The Graph` team:

> The owner is protocol governance, which is already trusted with broader powers, including upgrading the DisputeManager implementation. A delay on ŝetArbitrator`alone would not constrain a compromised owner. Arbitrator changes are governance decisions made transparently on chain, consistent with the legacy DisputeManager.

HAL-037 ACKNOWLEDGED

Full delegation slash strands thawing principal and blocks all delegator exits

Informational · 1.4 AO:A/AC:M/AX:H/R:N/S:U/C:N/A:M/I:N/D:M/Y:N

DESCRIPTION

Once the governor enables delegation slashing, which is a one-way switch with no disable path, a verifier calling `HorizonStaking.slash()` can consume the entire delegation pool balance. When this happens the pool's thawing token balance is scaled to zero and the pool's thawing nonce is incremented, which permanently invalidates every outstanding thaw request. Affected delegators have already had their pool shares burned by `HorizonStaking.undelegate()`, so their principal is unrecoverable through any path.

The pool is also left holding shares against a zero token balance. While any shares remain outstanding, `HorizonStaking.delegate()`, `HorizonStaking.undelegate()`, and `HorizonStaking.withdrawDelegated()` all revert on the invalid-pool state check, blocking every remaining delegator from exiting. The only route back to a usable state is for an outside party to call `HorizonStaking.addToDelegationPool()`, which credits tokens without minting shares and therefore functions as a donation to existing holders rather than a recovery for the delegators who were harmed. Both consequences are acknowledged in the contract documentation, but no protocol-native remedy exists.

RECOMMENDATION

A slash that zeroes the pool balance should settle outstanding positions at the time of the slash rather than leaving shares stranded against no backing. Provide a first-class recovery path that lets affected delegators exit the invalid pool state without depending on a third-party donation, and avoid invalidating in-flight thaw requests through a nonce increment without an accompanying settlement of the burned shares.

REMEDIATION COMMENT

ACKNOWLEDGED: The full-delegation-slash behavior was retained by the `The Graph` team. Pool tokens can be reduced to zero and thaw requests invalidated in `HorizonStaking.sol:407-428`, after which a pool with outstanding shares is rejected in `HorizonStaking.sol:856-870` until tokens are added. The following statement was provided by the `The Graph` team:

> This behavior is documented on ùndelegate.` Thawing delegation remains slashable by design, and in a full slash there is no remaining value to settle for either thawing or non-thawing positions. The invalid pool state contains no recoverable value, and the documented recovery path is to add tokens

> back to the pool. Delegation slashing is currently disabled on Arbitrum One, and a full delegation slash requires the provider's own stake to be fully slashed first.

HAL-038 ACKNOWLEDGED

Expired thaw requests remain locked behind an earlier request with a later maturity

Informational · 1.4 AO:A/AC:M/AX:M/R:N/S:U/C:N/A:L/I:N/D:L/Y:N

DESCRIPTION

Thaw request fulfillment traverses the request list in creation order and stops at the first entry whose maturity timestamp has not yet passed. Because the provision thawing period can be lowered through `HorizonStaking.setProvisionParameters()` followed by `HorizonStaking.acceptProvisionParameters()`, a request created after the reduction can mature earlier than one created before it. Traversal still stops at the older, later-maturing request, so the newer request stays locked even though its thawing period has fully elapsed.

This affects both provision thawing via `HorizonStaking.deprovision()` and delegation thawing via `HorizonStaking.withdrawDelegated()`, and supplying a request count does not skip the blocking head. The condition resolves on its own once the head matures, so no funds are lost, but tokens that are contractually withdrawable remain inaccessible for the remainder of the older request's period.

RECOMMENDATION

Fulfillment should release any request whose maturity timestamp has passed, independent of its position in the creation-ordered list. Skipping non-matured entries during traversal rather than terminating on them, or ordering the list by maturity, would resolve this without changing the accounting model.

REMEDIATION COMMENT

ACKNOWLEDGED: Creation-order thaw processing was retained by the `The Graph` team. Traversal was verified to stop at the first unmatured head in `HorizonStaking.sol:1001-1021` and `HorizonStaking.sol:1045-1060`; a later matured request can remain blocked. The following statement was provided by the `The Graph` team:

> This is documented behavior: thaw requests are processed in creation order, and the NatSpec notes that a shortened thawing period can leave expired requests behind an unexpired one. It is the same trade-off as the stake-claim ordering in FIND-047, chosen for bounded and predictable traversal. No funds are lost, and the delay is bounded by the original thawing period of the older request. For SubgraphService, thawing period changes only follow dispute period changes, which are infrequent governance actions.

HAL-039 ACKNOWLEDGED

Large eligibilityPeriod Setting Bypasses Oracle Access Control, Making All Unregistered Indexers Eligible for Rewards

Informational · 1.1 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:M/I:L/D:N/Y:N

DESCRIPTION

The `isEligible()` function in `RewardsEligibilityOracle` determines indexer eligibility via the check `block.timestamp < $.indexerEligibilityTimestamps[indexer] + $.eligibilityPeriod`. For unregistered indexers (never renewed by the oracle), `indexerEligibilityTimestamps[indexer] = 0`. If an operator with `OPERATOR_ROLE` sets `eligibilityPeriod >= block.timestamp` (approximately ≥ 1.75 billion as of 2025), the check becomes `block.timestamp < 0 + eligibilityPeriod` which evaluates to `true` ; making ALL unregistered indexers perpetually eligible regardless of oracle status. The `setEligibilityPeriod()` function has no upper bound validation on the input value.

RECOMMENDATION

Cap the maximum allowed value in `setEligibilityPeriod()` at a reasonable upper bound that is clearly less than current `block.timestamp`.

REMEDIATION COMMENT

ACKNOWLEDGED: The operator-controlled eligibility period was retained by `The Graph` team. No upper bound was added in `RewardsEligibilityOracle.sol:163-173`, and an unregistered indexer can be treated as eligible through `RewardsEligibilityOracle.sol:297-306` when the period exceeds the current timestamp. Validation can also be disabled by the operator in `RewardsEligibilityOracle.sol:205-213`. The following statement was provided by `The Graph` team:

> `OPERATOR_ROLE` is a trusted, governance-granted role that can already disable eligibility validation directly through `setEligibilityValidation(false)`, which makes every indexer eligible. A large eligibility period therefore grants no capability beyond what the role is explicitly entrusted with. Parameter changes by the operator are visible on chain through events.

HAL-040 ACKNOWLEDGED

RewardsManager Subgraph Availability Oracle Can Immediately Block All Subgraph Rewards via setMinimumSubgraphSignal

Informational · 1.1 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:M/I:L/D:N/Y:N

DESCRIPTION

The `setMinimumSubgraphSignal()` function in `RewardsManager` allows either the governor OR the `subgraphAvailabilityOracle` (SAO) to set the minimum curation signal required for a subgraph to earn rewards. There is no upper bound on the value that can be set. If the SAO is compromised or malicious, it can immediately call `setMinimumSubgraphSignal(type(uint256).max)`, which would cause `_getSubgraphRewardsState()` to classify all subgraphs as below the minimum signal threshold and reclaim (redirect) all their accruing rewards. This is a critical governance escalation path because the SAO is a less-privileged role (a stored address, not protected by timelock or multisig) that can have the same effect on protocol economics as denying every subgraph simultaneously.

RECOMMENDATION

Add an upper bound validation to `setMinimumSubgraphSignal()`.

REMEDIATION COMMENT

ACKNOWLEDGED: The oracle authority and unbounded minimum signal were retained by the `The Graph` team in `RewardsManager.sol:122-157`. The reviewed `SubgraphAvailabilityManager.sol:22-212` has a per-subgraph deny-or-allow route after votes and shows no route to set the minimum signal. The live oracle address and deployed code were not verified, so the submitted compromised-oracle call path was not established for the deployed system. The following statement was provided by the `The Graph` team:

> On Arbitrum One the subgraph availability oracle is the SubgraphAvailabilityManager`contract, which requires a threshold of independent oracle votes and can only deny or allow individual subgraphs. It has no path to call ŝetMinimumSubgraphSignal,` so in the deployed system only the governor can change the minimum signal. The governor is trusted with protocol parameters, and the procedure for changing this threshold is documented on the function.

HAL-041 ACKNOWLEDGED

Governor role administers itself with no fallback admin and no enforced delay

Informational · 1.0 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:M/I:N/D:N/Y:N

DESCRIPTION

The `BaseUpgradeable` contract sets `GOVERNOR_ROLE` as its own admin via `_setRoleAdmin(GOVERNORROLE, GOVERNOR_ROLE)`. OpenZeppelin's exposes `AccessControl` unconditionally `renounceRole(bytes32 role, address account)`, which allows any role holder to remove themselves from a role without restriction. If the last (or only) `GOVERNOR_ROLE` holder calls `renounceRole(GOVERNOR_ROLE, msg.sender)`, the `GOVERNOR_ROLE` becomes permanently self-locked: no account can grant `GOVERNOR_ROLE` because only a `GOVERNOR_ROLE` member can administer it, and there are none left. Since `DEFAULT_ADMIN_ROLE ( bytes32(0) )` is also not granted to any address (no fallback admin exists), there is no recovery mechanism whatsoever

RECOMMENDATION

Enforce minimum governor count before renouncement, and implement a two-step governance transfer: Before any governance transition, require a pending governor to accept the role.

REMEDIATION COMMENT

ACKNOWLEDGED: The self-administered governor role was retained by `The Graph` team. The initial governor role is granted and made its own admin in `BaseUpgradeable.sol:121-134`. No local guard prevents renunciation by the last holder. The stated proxy-admin recovery was not independently verified on-chain. The following statement was provided by `The Graph` team:

> The governor is protocol governance, and renouncing the role requires a deliberate action by governance itself. The contracts are upgradeable through a proxy admin controlled by governance, so the role can be restored through an upgrade in the unlikely event it is lost. The pause role's emergency revocation path explicitly cannot revoke the governor role.

HAL-042 ACKNOWLEDGED

Single-Step Ownership Transfer via OwnableUpgradeable in SubgraphService and DisputeManager Enables Irreversible Governance Loss

Informational · 1.0 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:M/I:N/D:N/Y:N

DESCRIPTION

Both `SubgraphService` and `DisputeManager` inherit from OpenZeppelin's `OwnableUpgradeable`, which provides a single-step `transferOwnership(address newOwner)` function. This function immediately transfers ownership without requiring the new owner to call `acceptOwnership()`. A typo, incorrect address, zero address, or compromised call can permanently transfer ownership to an uncontrolled account with no recovery mechanism. The contracts do not use `Ownable2StepUpgradeable`, which would provide a two-step "offer + accept" ownership transfer pattern.

RECOMMENDATION

Replace `OwnableUpgradeable` with `Ownable2StepUpgradeable` in both `SubgraphService` and `DisputeManager`. Consider overriding `renounceOwnership()` to revert with a custom error, preventing accidental permanent lockout.

REMEDIATION COMMENT

ACKNOWLEDGED: The inherited ownership transfer design was retained by the `The Graph` team. `OwnableUpgradeable` is inherited by `SubgraphService.sol:39-50` and `DisputeManager.sol:50-57`; no two-step transfer is added by either contract. Recovery through a proxy upgrade depends on the proxy administrator. The following statement was provided by the `The Graph` team:

> Both contracts are owned by protocol governance, and ownership changes are deliberate governance transactions. The contracts are upgradeable through a proxy admin controlled by governance, so ownership can be restored through an upgrade in the event of an erroneous transfer.

HAL-043 ACKNOWLEDGED

PAUSE_ROLE can revoke critical operational roles via emergencyRevokeRole, enabling DoS of agreement management

Informational · 1.0 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:M/I:N/D:N/Y:N

DESCRIPTION

The `RecurringAgreementManager.emergencyRevokeRole()` function allows a `PAUSE_ROLE` holder to revoke any role except `GOVERNOR_ROLE`. This creates an unintended privilege escalation path where a single `PAUSE_ROLE` account, which is intended only for emergency pausing,can permanently disrupt protocol operations by revoking critical operational roles such as `OPERATOR_ROLE`, `AGREEMENT_MANAGER_ROLE`, `COLLECTOR_ROLE`, and `DATA_SERVICE_ROLE`. The role hierarchy is:

- `GOVERNOR_ROLE` (admin of `PAUSE_ROLE`, `OPERATOR_ROLE`).
- `OPERATOR_ROLE` (admin of `AGREEMENT_MANAGER_ROLE`).
- `PAUSE_ROLE` (intended only for pause/unpause).

The `PAUSE_ROLE` is explicitly designed to be granted to automated monitoring systems or emergency responders. A compromised `PAUSE_ROLE` account can revoke `OPERATOR_ROLE`, which is the admin of `AGREEMENT_MANAGER_ROLE`. This effectively locks out agreement lifecycle operations without any path to recovery except through `GOVERNOR_ROLE` re-granting.

RECOMMENDATION

Restrict `emergencyRevokeRole()` to only revoke `PAUSE_ROLE` itself (self-revocation), or limit it to a predefined set of safe roles that don't affect protocol operations (e.g., only `PAUSE_ROLE` members). Alternatively, rename and re-scope the function to `emergencyClearPauseGuardians()` that only removes `PAUSE_ROLE` members, separating it from general role revocation. Add an explicit exclusion list: `require(role != OPERATOR_ROLE && role != COLLECTOR_ROLE && role != DATA_SERVICE_ROLE, ...)` to prevent disruption of core operational roles.

REMEDIATION COMMENT

ACKNOWLEDGED: The emergency revocation authority was retained by `The Graph` team. `GOVERNOR_ROLE` is excluded in `RecurringAgreementManager.sol:395-400`, but operational roles can still be revoked by a pause guardian. Restoration by governance is supported by the role-admin

assignments in `RecurringAgreementManager.sol:246-250` and `BaseUpgradeable.sol:121-134`. The following statement was provided by `The Graph` team:

> This is intentional and documented as the first step of the contract's emergency escalation ladder: the pause guardian can disable a specific compromised actor, such as an operator or collector, without waiting for governance. It grants no capability beyond the guardian's existing ability to pause the contract, which is a broader disruption. Any revocation is recoverable by the governor re-granting the role. Restricting it to the pause role would remove its purpose.

HAL-044 ACKNOWLEDGED

Unbounded max POI staleness allows reward confiscation or freshness bypass

Informational · 1.0 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:M/I:N/D:N/Y:N

DESCRIPTION

`AllocationManager._setMaxPoiStaleness()` accepts any value with no minimum or maximum bound. The stored value directly controls which path `AllocationHandler.presentPOI()` takes, since `Allocation.isStale()` compares the time since the later of allocation creation and last POI presentation against the threshold.

When set to zero, an allocation becomes stale as soon as any time has elapsed since its last POI presentation, which in practice means from the block after creation onward rather than within the creation block itself, because the comparison is strict. Every presentation from that point routes through the stale condition, which calls `RewardsManager.reclaimRewards()` instead of `RewardsManager.takeRewards()`, redirecting all indexing rewards away from indexers across the network. `SubgraphService.closeStaleAllocation()` also becomes callable against every non altruistic allocation.

When set to a near maximum value, no allocation is ever considered stale regardless of how long it has gone without presenting a proof. This disables the freshness enforcement that protects the protocol from indexers collecting rewards on allocations they are no longer actively serving.

RECOMMENDATION

Add a minimum and a maximum bound to `AllocationManager._setMaxPoiStaleness()` so the value cannot be set to zero or to a value that neuters freshness enforcement. The bounds should reflect the operationally reasonable range for POI presentation frequency.

REMEDIATION COMMENT

ACKNOWLEDGED: The unrestricted staleness parameter was retained by the `The Graph` team. The owner setter in `SubgraphService.sol:355-357` reaches a direct assignment in `AllocationManager.sol:217-220`. No on-chain minimum or maximum constrains the value. The following statement was provided by the `The Graph` team:

> `maxPOIStaleness` is a governance-controlled parameter, and the owner is protocol governance. Values are chosen through the governance process, and a misconfiguration can be corrected with a single transaction. We prefer not to hardcode bounds that would constrain legitimate future adjustments.

HAL-045 ACKNOWLEDGED

Unbounded stake to fees ratio bricks fee collection

Informational · 1.0 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:M/I:N/D:N/Y:N

DESCRIPTION

`SubgraphService._setStakeToFeesRatio()` rejects zero but enforces no upper bound. The value is a raw multiplier applied during both query fee and indexing fee collection, where the locked stake amount is computed as the collected tokens multiplied by the ratio.

`SubgraphService.collect()` locks stake equal to `tokensCollected` multiplied by `stakeToFeesRatio` against the indexer's provision for the dispute period. When the ratio is set to a large value, the lock amount exceeds the indexer's available provision and the lock call reverts, making every fee collection transaction fail. The naming convention shares the ratio and cut terminology used elsewhere in the codebase for PPM-scaled values, so an owner treating it as a PPM value and setting it to 1000000 would multiply every collection by a million, permanently bricking fee collection for all indexers until the value is corrected.

RECOMMENDATION

Add an upper bound to `SubgraphService._setStakeToFeesRatio()` that reflects the intended unit and prevents a value that would make the lock amount exceed any reasonable provision size. Document the expected unit explicitly in the NatSpec so the ratio is not confused with the PPM-scaled cuts used throughout the rest of the codebase.

REMEDIATION COMMENT

ACKNOWLEDGED: The stake-to-fees unit was documented as a plain multiplier by the `The Graph` team in `ISubgraphService.sol:258-266`. The setter still accepts any nonzero value, and fee collection still multiplies collected tokens by that value in `SubgraphService.sol:695-700` and `SubgraphService.sol:792-812`. A high value can still revert collection. The following statement was provided by the `The Graph` team:

> The stake-to-fees ratio is a governance-controlled parameter, and the owner is protocol governance. The deployed value is 10, and a misconfiguration only causes collections to revert until corrected with a single transaction, without putting funds at risk. We agree the unit should be explicit and will document that the ratio is a plain multiplier, not a PPM value.

HAL-046 SOLVED

NatSpec Unit Mismatch in Allocation.isStale() Documents staleThreshold as Blocks but Compares Against Seconds, Risking Governance Misconfiguration

Informational · 0.8 AO:A/AC:L/AX:H/R:N/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

The `isStale()` function in the `Allocation` library has a unit mismatch in its NatSpec documentation: the `@param staleThreshold` is documented as "time in blocks" but the function computes `timeSinceLastPOI` in **seconds** using `block.timestamp` subtraction. This means `staleThreshold` is effectively a seconds value despite the documentation claiming it is in blocks. The function is internally consistent (all timestamps use `block.timestamp` which is in seconds), but the misleading parameter documentation could cause governance or developers to inadvertently set `maxPOIStaleness` in block counts rather than seconds when interacting with `setMaxPOIStaleness()`, leading to either premature or indefinitely delayed forced-close of stale allocations.

RECOMMENDATION

Fix the NatSpec documentation in `Allocation.isStale()` to accurately state that `staleThreshold` is in seconds.

REMEDIATION COMMENT

Addressed in: 1940caf24e833227b830d975e66604d74cacd116

SOLVED: The stale-threshold unit was corrected to seconds in the NatSpec by the `The Graph` team. The corrected unit was verified against the timestamp comparison in `Allocation.sol:133-142` and the setter documentation in `ISubgraphService.sol:268-273`. Deployment of the documentation change was not verified. The following statement was provided by the `The Graph` team:

> Confirmed. The setter documents `maxPOIStaleness` in seconds, but the library NatSpec is wrong. We will correct it.

HAL-047 ACKNOWLEDGED

Missing maxInitialTokens Overflow Validation in RecurringCollector Allows Permanent DoS on First Collection

Informational · 0.5 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

In `RecurringCollector._requireValidCollect()`, the `maxTokens` computation adds `maxInitialTokens` to `maxOngoingTokensPerSecond * collectionSeconds`:

SOL

```
uint256 maxTokens = _agreement.maxOngoingTokensPerSecond * _collectionSeconds; maxTokens +=
_agreement.lastCollectionAt == 0 ? _agreement.maxInitialTokens : 0;
```

The validation at offer/acceptance time `_requireValidTerms()` only performs an overflow guard for the rate multiplication:

SOL

```
_maxOngoingTokensPerSecond * _maxSecondsPerCollection * 1024; // overflow check only
```

This ensures `rate * collectionSeconds <= type(uint256).max / 1024`, but `*maxInitialTokens` is not included in this overflow check. A payer can set `maxInitialTokens` near `type(uint256).max`, causing the addition `maxTokens + maxInitialTokens` to overflow and revert on the first collection attempt. In Solidity 0.8, the addition overflows and reverts, making the agreement permanently uncollectable for its first collection window.

RECOMMENDATION

Add an overflow-safe check for `maxInitialTokens` in `_requireValidTerms()`.

REMEDIATION COMMENT

ACKNOWLEDGED: The term validation without a combined initial-token bound was retained by the `The Graph` team. Only the ongoing-rate product was checked in `RecurringCollector.sol:953-966`, while `maxInitialTokens` is added during first collection in `RecurringCollector.sol:979-1003`; an extreme value can still cause overflow. The following statement was provided by the `The Graph` team:

> Agreement terms are offered by the payer and must be explicitly accepted by the service provider. An absurd `maxInitialTokens` close to the integer limit is visible at acceptance time, and providers are expected to validate terms before accepting. No funds are at risk, the payer gains nothing, and the provider can cancel the agreement. We will consider adding the bound to term validation in a future RecurringCollector upgrade.

HAL-048 ACKNOWLEDGED

Eligibility parameter setters lack upper bounds causing potential revert in isEligible

Informational · 0.5 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

`RewardsEligibilityOracle.setEligibilityPeriod()` and `RewardsEligibilityOracle.setOracleUpdateTimeout()` accept any `uint256` value with no upper bound validation. `RewardsEligibilityOracle.isEligible()` performs checked addition on these values: `lastOracleUpdateTime` plus `oracleUpdateTimeout` on the timeout check, and `indexerEligibilityTimestamps` plus `eligibilityPeriod` on the per-indexer check.

When either parameter is set to a value near `type(uint256).max`, the addition overflows under Solidity 0.8 checked arithmetic and reverts. For the timeout check, this occurs once any oracle push has set `lastOracleUpdateTime` to a nonzero value. For the eligibility check, this occurs for any indexer whose eligibility has been renewed. The documented fail-open behavior, where `RewardsEligibilityOracle.isEligible()` returns true after a timeout silence, instead becomes a hard revert, which is the opposite of the intended fail-safe posture.

RECOMMENDATION

Add a reasonable upper bound in `RewardsEligibilityOracle.setEligibilityPeriod()` and `RewardsEligibilityOracle.setOracleUpdateTimeout()` to ensure the checked additions in `RewardsEligibilityOracle.isEligible()` cannot overflow for any realistic timestamp.

REMEDIATION COMMENT

ACKNOWLEDGED: The unbounded eligibility parameter setters were retained by `The Graph` team. Any `uint256` is still accepted for both periods in `RewardsEligibilityOracle.sol:163-193`, while checked additions in `RewardsEligibilityOracle.sol:297-306` can revert at extreme values. Operator correction and governor removal of the oracle remain available through `RewardsManager.sol:218-234`. The following statement was provided by `The Graph` team:

> These parameters are set by the trusted OPERATOR_ROLE and are only reachable through an obvious misconfiguration. If it occurred, it is recoverable in a single transaction, either by correcting the parameter or by governance removing the oracle from RewardsManager. Rewards are not lost in the meantime, since reverted collections leave rewards claimable.

HAL-049 ACKNOWLEDGED

Governor role is self-administering with no enforced timelock or separation

Informational · 0.5 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

`BaseUpgradeable.__BaseUpgradeable_init_unchained()` sets `GOVERNOR_ROLE` as its own role admin. This means any address holding the governor role can grant itself or any other address the governor role, revoke other governors, and manage all downstream roles including `PAUSE_ROLE` and `OPERATOR_ROLE`, with no on-chain delay or multi-signature requirement.

All contracts in the issuance package inherit from `BaseUpgradeable`, so this self-admin pattern applies to `IssuanceAllocator`, `DirectAllocation`, `RecurringAgreementManager`, and `RewardsEligibilityOracle`. A compromised governor key can unilaterally reconfigure issuance rates, allocation targets, escrow parameters, and eligibility settings in a single transaction with no time for detection or response.

RECOMMENDATION

Ensure the governor address is a multi-signature wallet or governance contract with an enforced timelock for privileged operations. Consider separating the role-admin capability from the operational governor functions so that role hierarchy changes require a higher threshold than routine parameter updates.

REMEDIATION COMMENT

ACKNOWLEDGED: The self-administered governor role was retained by `The Graph` team. `GOVERNOR_ROLE` is made its own admin, and no timelock is enforced for role changes in `BaseUpgradeable.sol:121-134`. The stated multisignature threshold was not independently verified on-chain. The following statement was provided by `The Graph` team:

> The governor role in the deployed issuance contracts is held by the protocol governor, a 4-of-7 multisig, which is the same governance model used across all protocol contracts. Governance actions are executed through that multisig's process. See also our response to FIND-012.

HAL-050 SOLVED

Payer cancellation of an accepted agreement is a request to the data service rather than a state change

Informational · 0.5 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

A payer invoking `RecurringCollector.cancel()` with the active scope set does not itself change the agreement state. It forwards the request to the data service through `cancelIndexingAgreementByPayer()` and returns. The state transition happens in the separate cancel entry point that requires the caller to be the recorded data service.

For the deployed data service the round trip completes. `SubgraphService.cancelIndexingAgreementByPayer()` reaches `IndexingAgreement.cancelByPayer()`, which authorizes the caller against the payer and calls `_cancel()`, whose final statement calls back into `RecurringCollector.cancel(agreementId, cancelBy)` as the data service. The agreement moves to a cancelled state in the same transaction. There is no defect in the current wiring.

What the code does not express is that this outcome is a property of the data service implementation rather than of the collector. `RecurringCollector` is generic and any registered data service can hold agreements. A data service that accepts the forwarded call and does not complete the round trip would leave the agreement accepted and collectable while the payer's intent is recorded only in an event. The NatSpec notes that this cancellation path targets the payer side and that extending it to other callers is deferred, but it does not state that the active scope is a request whose fulfilment depends on the callee.

RECOMMENDATION

Document at the cancellation entry point that the active scope forwards to the data service and that the state transition is performed there, so anyone building a new data service understands the obligation they are taking on. If the collector should not depend on data service behaviour for a payer initiated termination, give the payer a direct path that takes effect after a grace period when a forwarded request has not been processed.

REMEDIATION COMMENT

Addressed in: 1940caf24e833227b830d975e66604d74cacd116

SOLVED: The data-service callback duty for active payer cancellation was documented by the `The Graph` team in `IAgreementCollector.sol:134-152` and `IDataServiceAgreements.sol:13-21`. The reviewed `SubgraphService.sol:456-472` path completes the callback through

`IndexingAgreement.sol:659-679`. Other data-service implementations were not reviewed and must satisfy the documented callback duty. The following statement was provided by the `The Graph` team:

> Routing payer cancellation through the data service is intentional, since the data service owns the agreement lifecycle and must update its own state alongside the collector. As noted, the deployed SubgraphService completes the round trip atomically. We agree that the obligation should be explicit and will document it on both the collector's cancel entry point and the `IDataServiceAgreements` interface.

HAL-051 ACKNOWLEDGED

Thawed token view and the executable fulfillment path order maturity and nonce checks differently

Informational · 0.5 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

`HorizonStakingBase.getThawedTokens()` nests the maturity check inside the nonce check, so a request whose nonce is stale is skipped and traversal continues past it. `HorizonStaking._fulfillThawRequest()` checks maturity first and stops on the first unmatured request whatever its nonce.

They diverge on exactly one arrangement, an unmatured stale nonce request sitting ahead of a matured current nonce one. The view counts the later request, the transaction releases nothing. Every other combination agrees. Since the view is external and integration facing, a keeper or interface consulting it before withdrawing gets a number the transaction will not deliver.

Reaching that arrangement needs two things at once. The nonce increments only in `HorizonStaking.slash()`, and only when the slash leaves thawing shares with zero thawing tokens, which means a slash down to dust rather than an ordinary one. Separately the thawing period must have been shortened, which `HorizonStaking.setProvisionParameters()` only stages and the verifier must accept through `HorizonStaking.acceptProvisionParameters()`, with `ProvisionManager` revalidating the range.

RECOMMENDATION

Hoist the maturity test out of the nonce branch in `HorizonStakingBase.getThawedTokens()` so both traversals follow the same rule, and change both together if either is changed later.

REMEDIATION COMMENT

ACKNOWLEDGED: The thawed-token view mismatch was retained by the `The Graph` team. A stale nonce is checked before maturity in `HorizonStakingBase.sol:201-217`, while maturity is checked first in `HorizonStaking.sol:1045-1065`; the view can report tokens that a withdrawal will not release. The following statement was provided by the `The Graph` team:

> Confirmed. The divergence only affects the view in an edge case that requires both a slash leaving thawing tokens at zero and a subsequent thawing period reduction. Funds are unaffected, and the executable path is authoritative. We will align the view with the fulfillment logic in a future HorizonStaking upgrade.

Disclaimer

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.